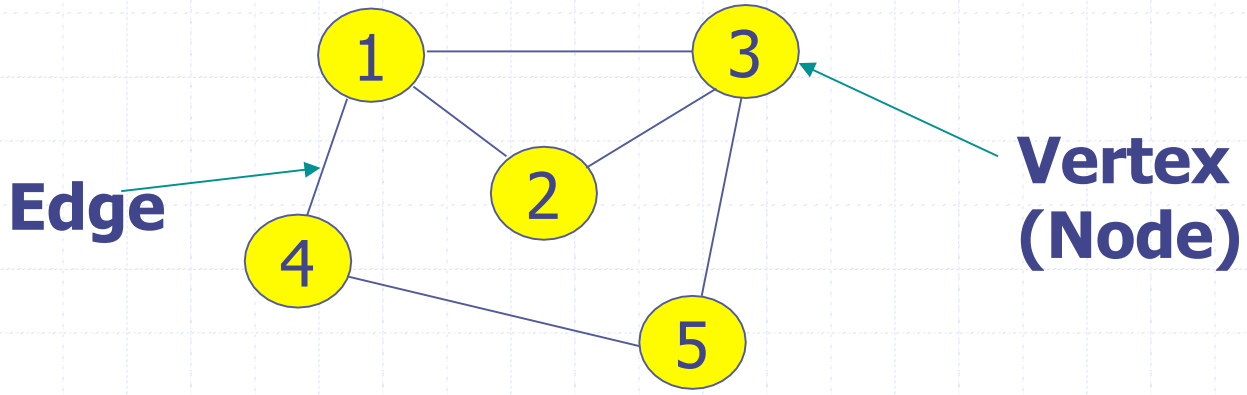
CS1312
Algorithms Design and Analysis

Graphs

What is Graph?

- A Graph is a data structure which consists of a set of *vertices*, and a set of *edges* that connect (some of) them.
- That is, **G = (V, E),** Where V - set of vertices, E - set of edges

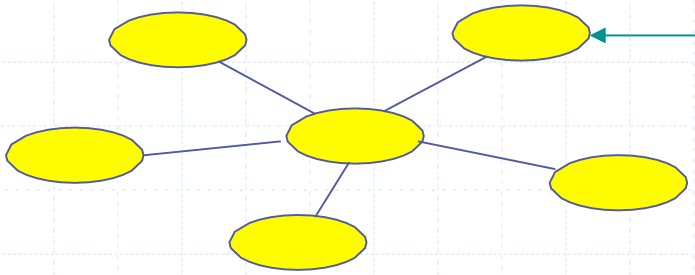


V = {1, 2, 3, 4, 5}

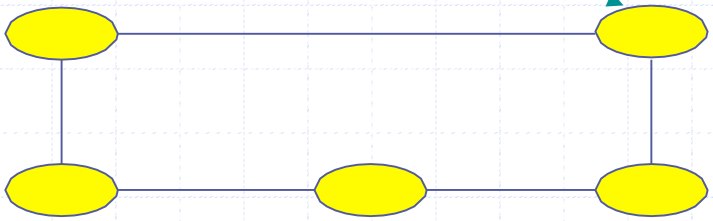
E = { (1,2), (1,3), (1,4), (2,3), (3,5), (4,5) }

Applications

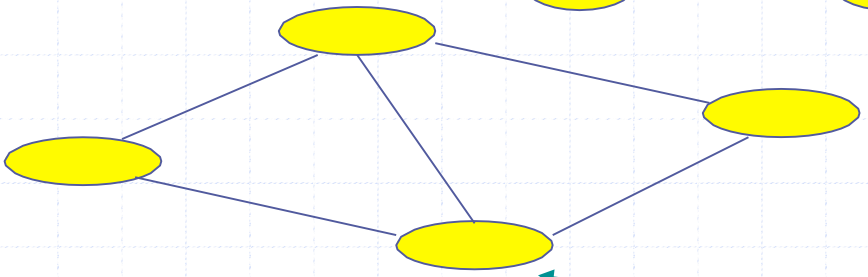
- Computer Networks



Resistor/Inductor/…



- Electrical Circuits



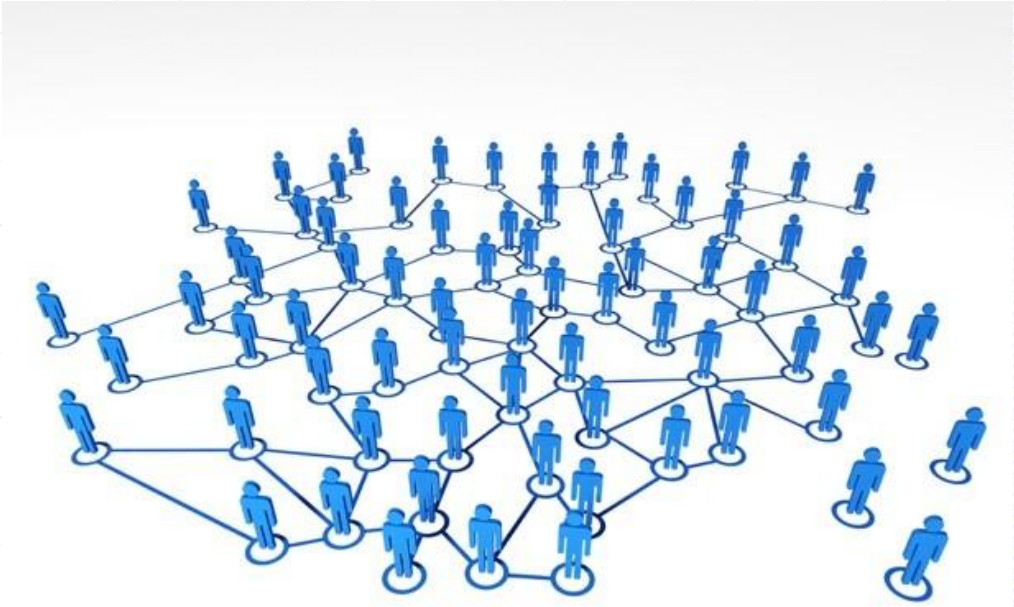
- Road Map

City

Why graphs are important?

Graphs are used a lot in computer science

- Social network (Facebook, LinkedIn)



Why graphs are important?

Graphs are used a lot in computer science

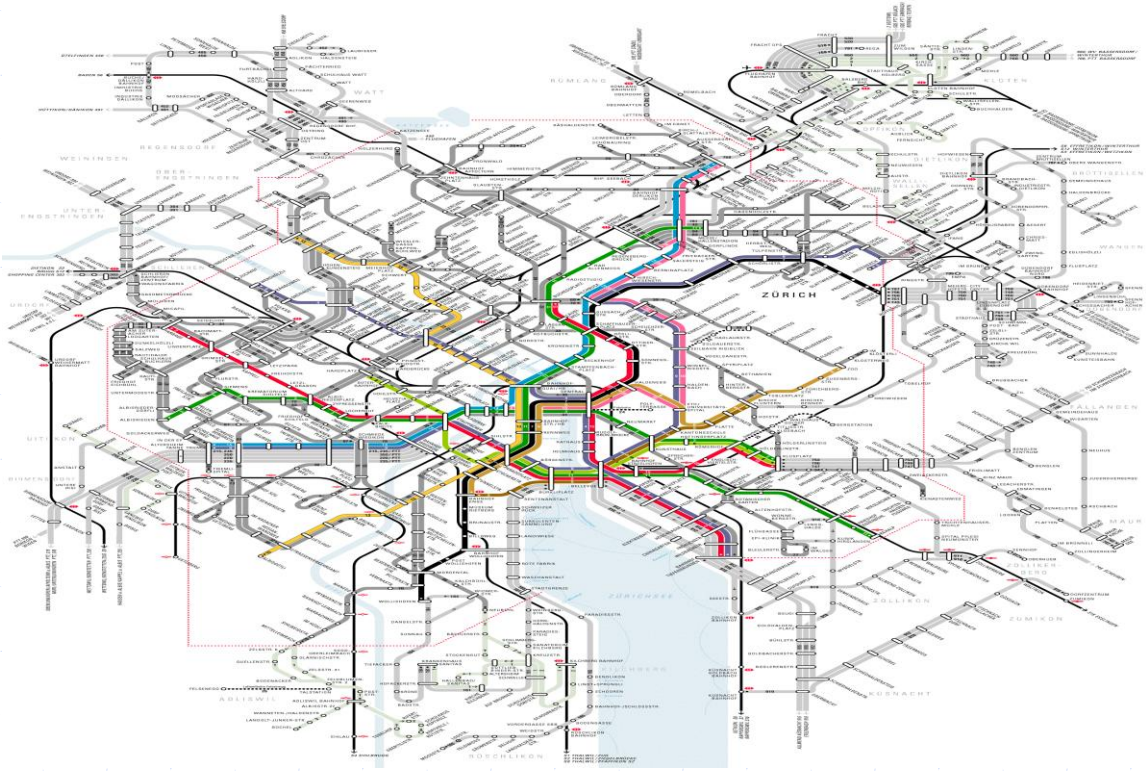
- Computer networks



Why graphs are important?

Graphs are used a lot in computer science

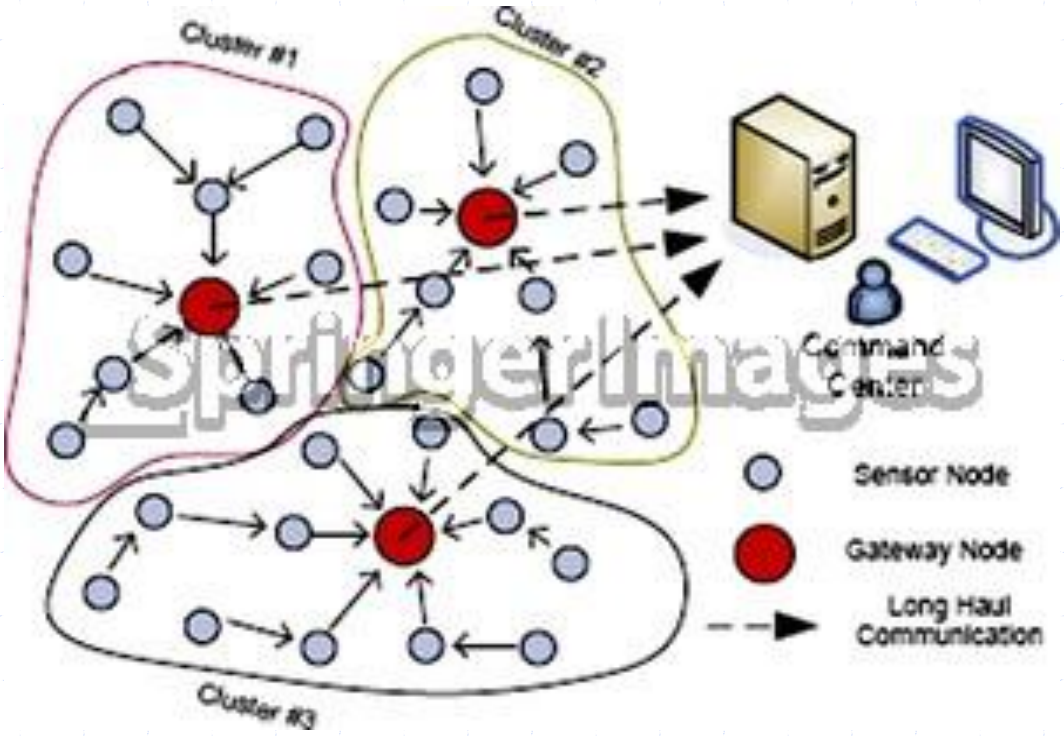
- Transportation network



Why graphs are important?

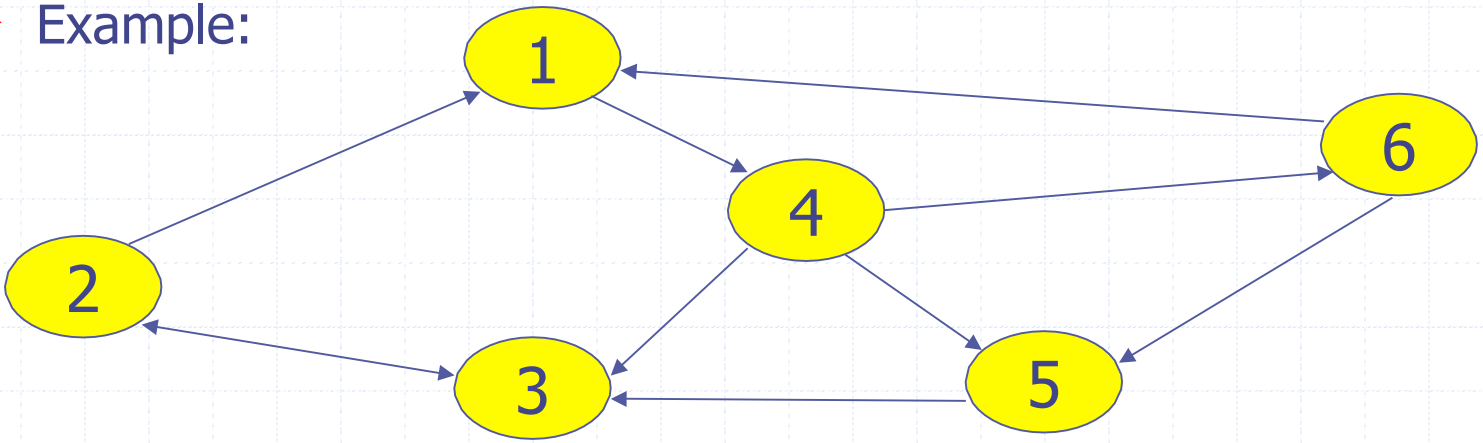
Graphs are used a lot in computer science

- Wireless sensors



Graph Categorization

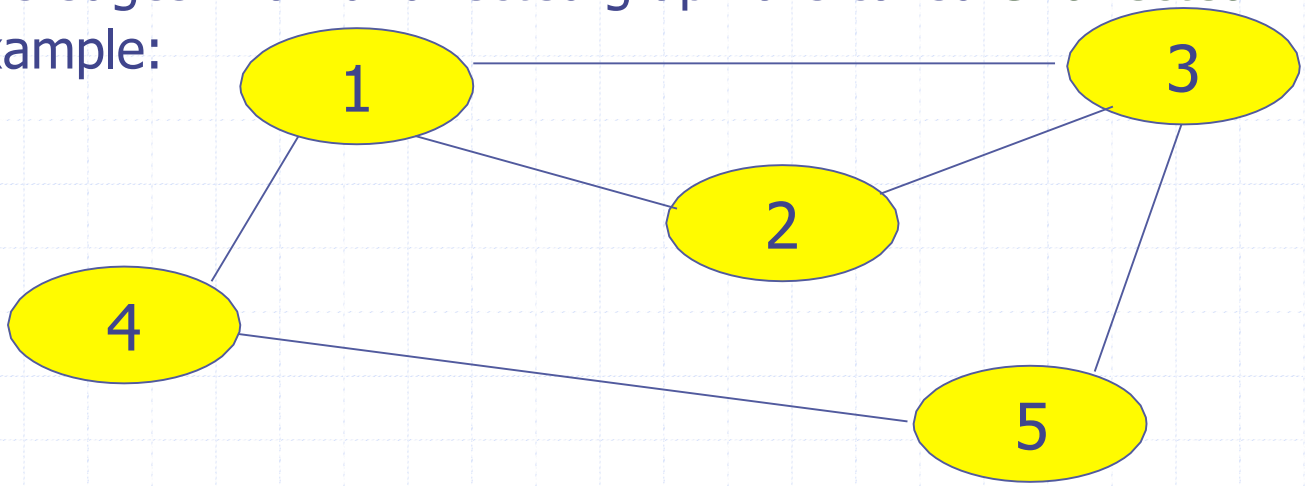
- A *Directed Graph* or *Digraph* is a graph where each edge has a direction
- The edges in a digraph are called *Arcs* or *Directed Edges*
- Example:



- G = (V, E), where V = {1, 2, 3, 4, 5, 6} and E = {(1,4), (2,1), (2,3), (3,2), (4,3), (4,5), (4,6), (5,3), (6,1), (6,5)}
- (1, 4) = 1→4 where 1 is the *tail* and 4 is the *head*

Graph Categorization (cont.)

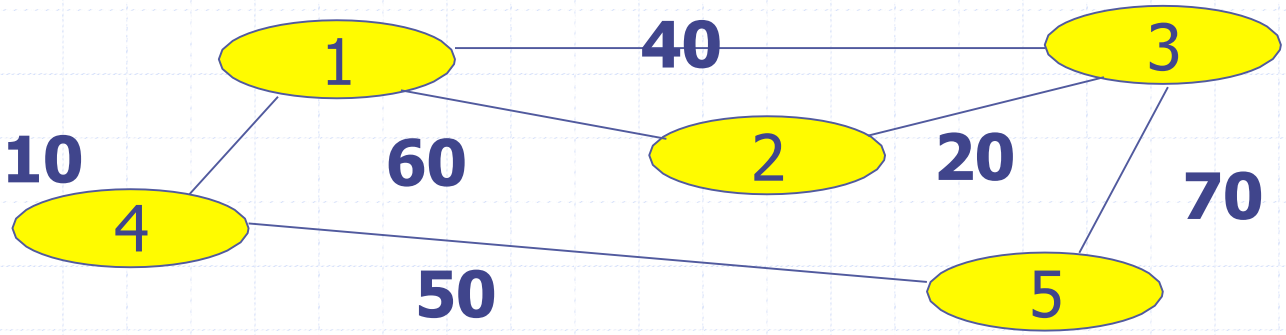
- An *Undirected Graph* is a graph where the edges have no directions
- The edges in an undirected graph are called *Undirected Edges*
- Example:



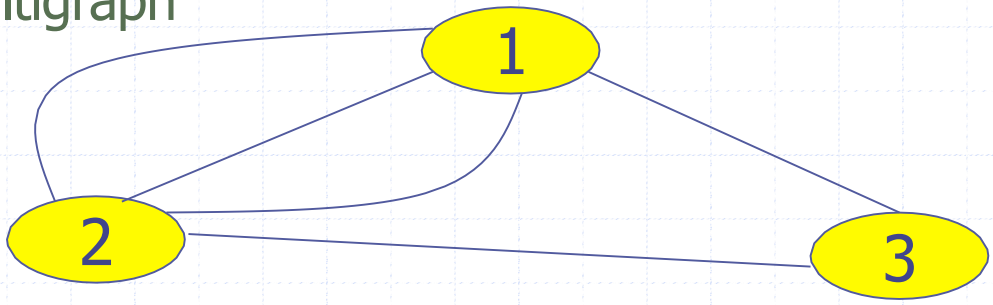
- G = (V, E), where V = {1, 2, 3, 4, 5} and E = {(1,2), (1,3), (1,4), (2,3), (3,5), (4,5)}

Graph Categorization (cont.)

- A Weighted Graph is a graph where all the edges are assigned weights.

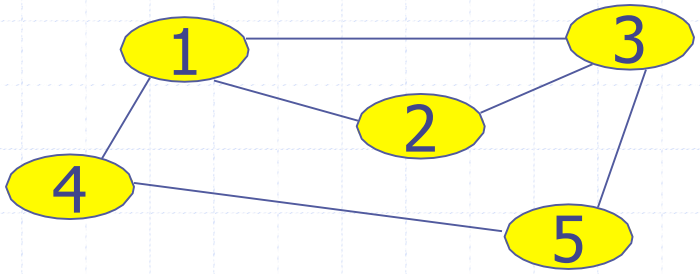


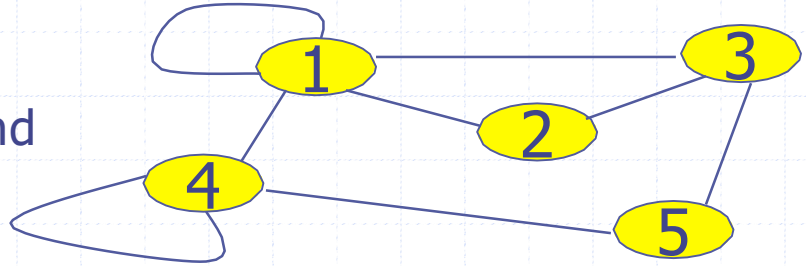
- If the same pair of vertices have more than one edge, that graph is called a Multigraph



Graph Terminology

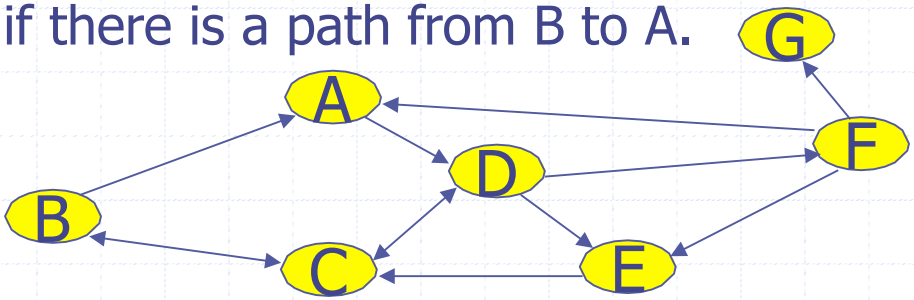
- Adjacent vertices: If (i,j) is an edge of the graph, then the nodes i and j are adjacent.
- edge (i,j) is Incident to vertices i *and j.*
- Vertices 2 and 5 are *not* adjacent
- Loop or self edges: An edge (i,i) is called a self edge or a loop.
  - (1,1) and (4,4) are self edges
- A graph with no self edges and multiple edges is called a "simple graph".



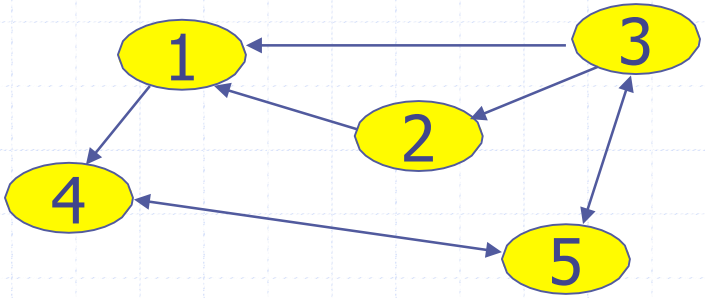


Graph Terminology (cont.)

- Path: A sequence of edges in the graph.
- There can be more than one path between two vertices.
- Vertex A is reachable from B if there is a path from B to A.
- E.g.: Paths from B to D
  - B, A, D
  - B, C, D

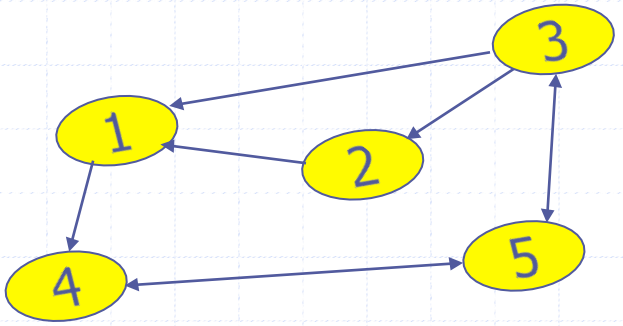


- Simple Path: A path where all the vertices are distinct.
  - E.g.: 1,4,5,3 is a simple path.
  - but 1,4,5,4 is not a simple path.



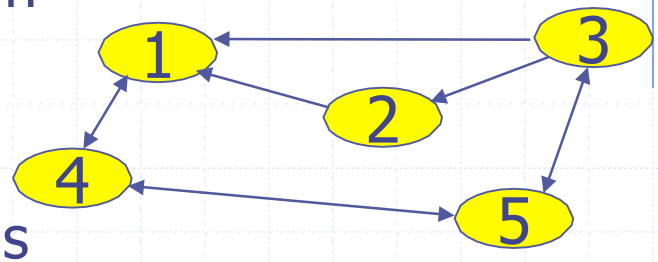
Graph Terminology (cont.)

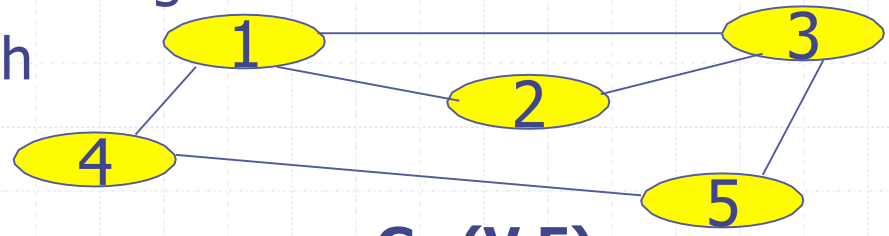
- Length: Sum of the lengths of the edges on the path.
  - Length of the path 1,4,5,3 is **3**
- Circuit: A path whose first and last vertices are the same.
  - The path 3,2,1,4,5,3 is a circuit
- Cycle: A circuit where all the vertices are distinct except for the first (and the last) vertex.
  - 1,4,5,3,1 is a cycle, but **1,4,5,4,1** is not a cycle.
- Hamiltonian Cycle: A Cycle that contains all the vertices of the graph.
  - 1,4,5,3,2,1 is a Hamiltonian Cycle.



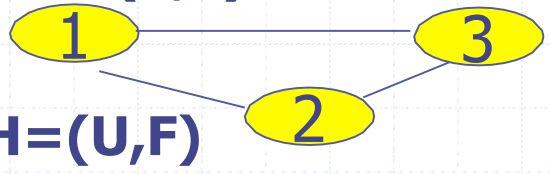
Graph Terminology (cont.)

- Degree of a Vertex : In an undirected graph, the no. of edges incident to the vertex
- In-degree: The no. of edges entering the vertex in a digraph
- Out-Degree: The no. of edges leaving the vertex in a digraph
  - In-degree of 1 is **3**
  - Out-degree of 1 is **1**
- A Subgraph of graph G=(V,E) is a graph H=(U,F) such that U Є V and F Є E





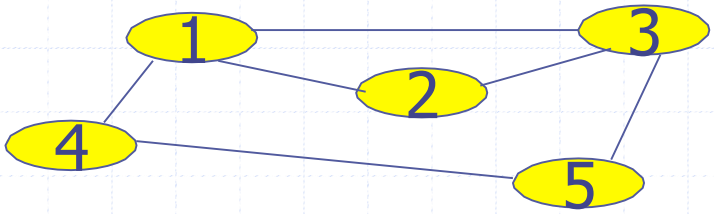
G=(V,E)



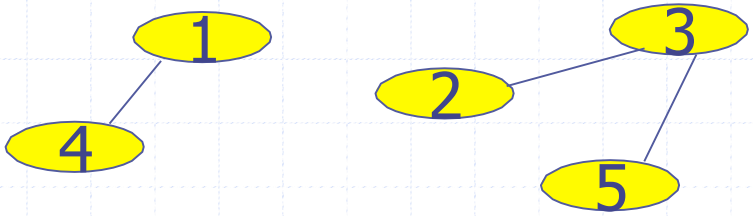
H=(U,F)

Graph Terminology (cont.)

- A graph is said to be Connected if there is at least one path from every vertex to every other vertex in the graph.

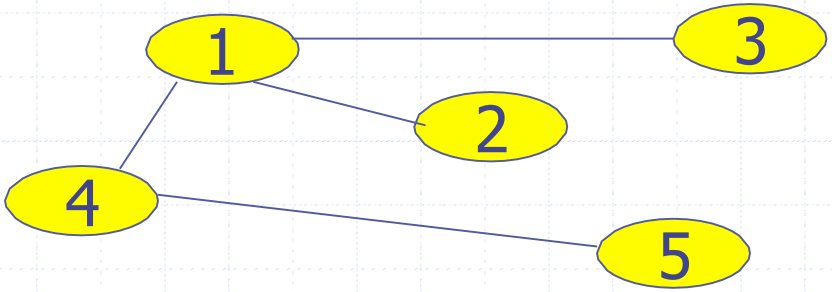


Connected

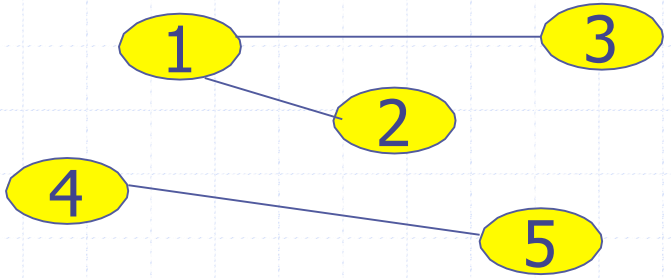


Unconnected

- Tree: A connected undirected graph that contains no cycles.
- Forest: A graph that does not contain a cycle.



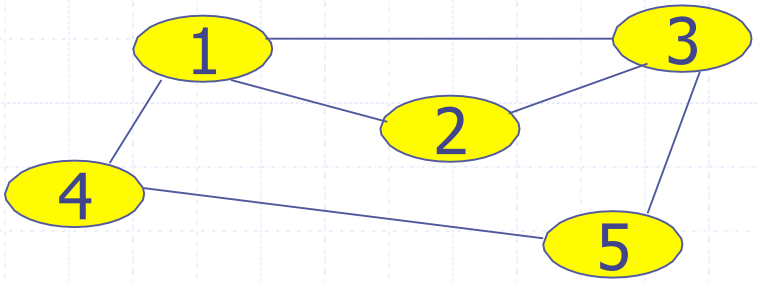
Tree



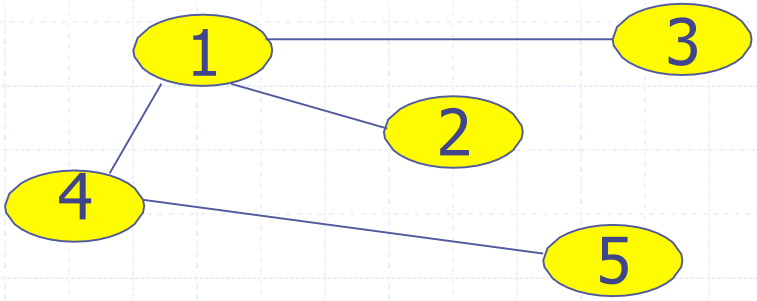
Forest

Graph Terminology (cont.)

- The Spanning Tree of a Graph G is a subgraph of G that is a tree and contains all the vertices of G.



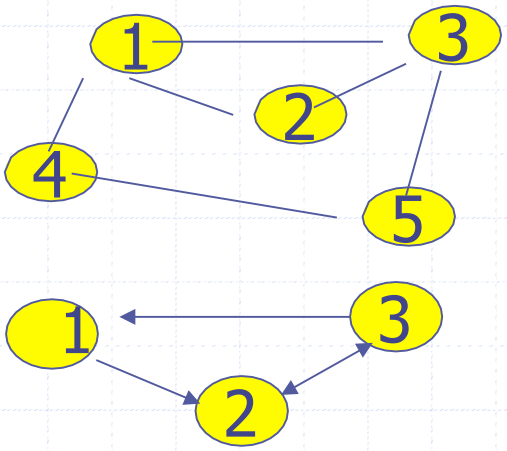
Graph



Spanning Tree

Representation of Graphs

- Adjacency Matrix (A)
  - The Adjacency Matrix $A=(a_{i,j})$ of a graph G=(V,E) with n nodes is an nXn matrix
  - Each element of A is either 0 or 1, depending on the adjacency of the nodes
  - $a_{ij} = 1$, if $(i,j) \in E$,
    $= 0$, otherwise
- Example: Find the adjacency matrices of the following graphs.

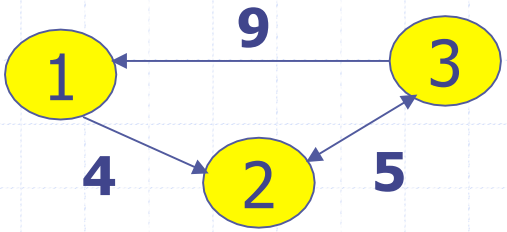


	1	2	3	4	5
1	0	1	1	1	0
2	1	0	1	0	0
3	1	1	0	0	1
4	1	0	0	0	1
5	0	0	1	1	0

0	1	0
0	0	1
1	1	0

Representation of Graphs (cont.)

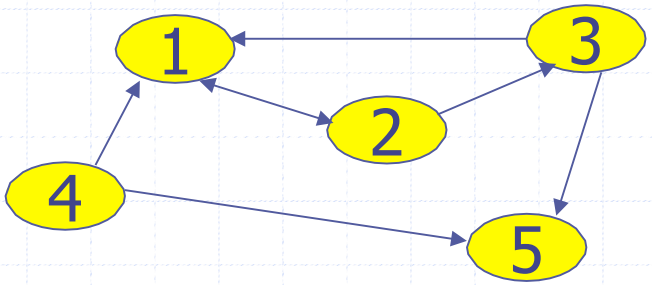
- *Adjacency Matrix* of a *Weighted Graph*
  - The weight of the edge can be shown in the matrix when the vertices are adjacent
  - A nil value (0 or ∞) depending on the problem is used when they are not adjacent
  - Example: To find the minimum distance between nodes...

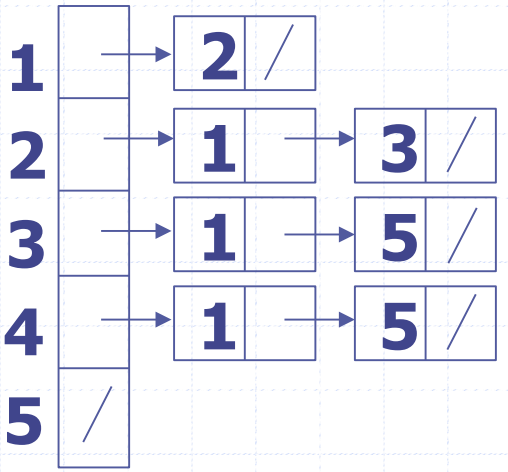


∞	4	∞
∞	∞	5
9	5	∞

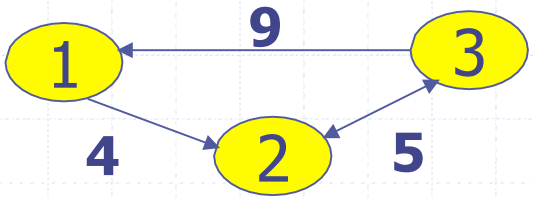
Representation of Graphs (cont.)

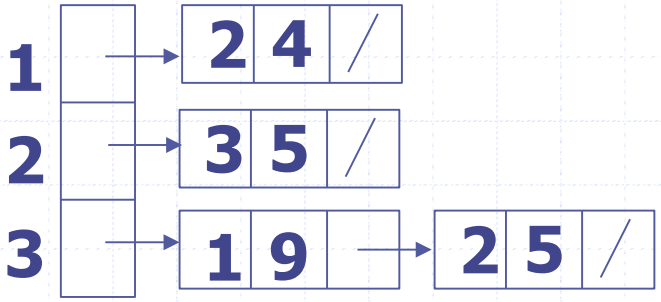
- *Adjacency List*
  - An Adjacency list is an array of lists, each list showing the vertices a given vertex is adjacent to.





- *Adjacency List* of a *Weighted Graph*
  - The weight is included in the list





Graph Searching

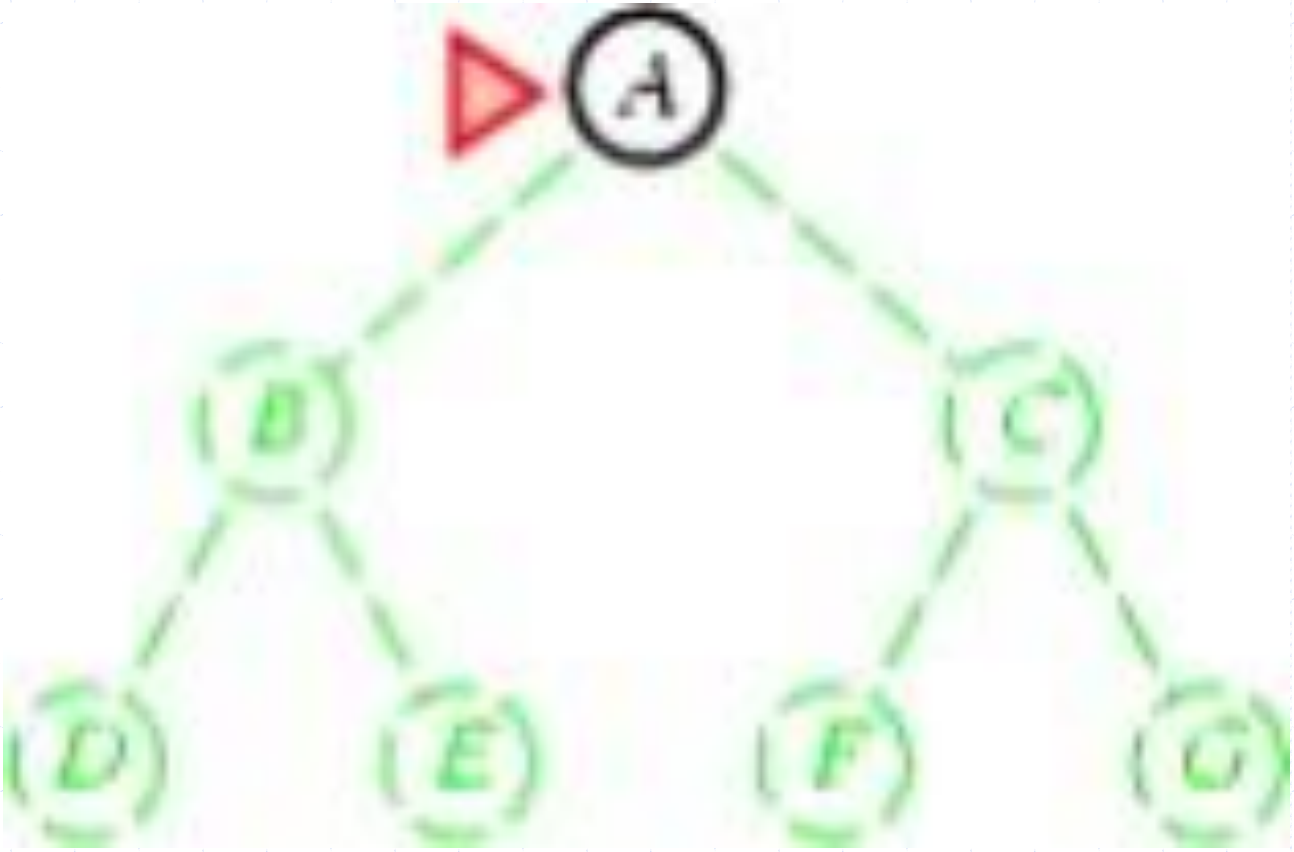
- Given: a graph G = (V, E), directed or undirected
- Goal: methodically explore every vertex and every edge
- Ultimately: build a tree on the graph
  - Pick a vertex as the root
  - Choose certain edges to produce a tree
  - Note: might also build a forest if graph is not connected

Breadth first Search (BFS)

- "Explore" a graph
  - One vertex at a time.
  - Expand frontier of explored vertices across the breadth of the frontier.
- Associate vertex with "colors" to guide the algorithm
  - White vertices have not been discovered
    - All vertices start out white
  - Grey vertices are discovered but not fully explored
    - They may be adjacent to white vertices
  - Black vertices are discovered and fully explored
    - They are adjacent only to black and gray vertices
- Explore vertices by scanning adjacency list of grey vertices

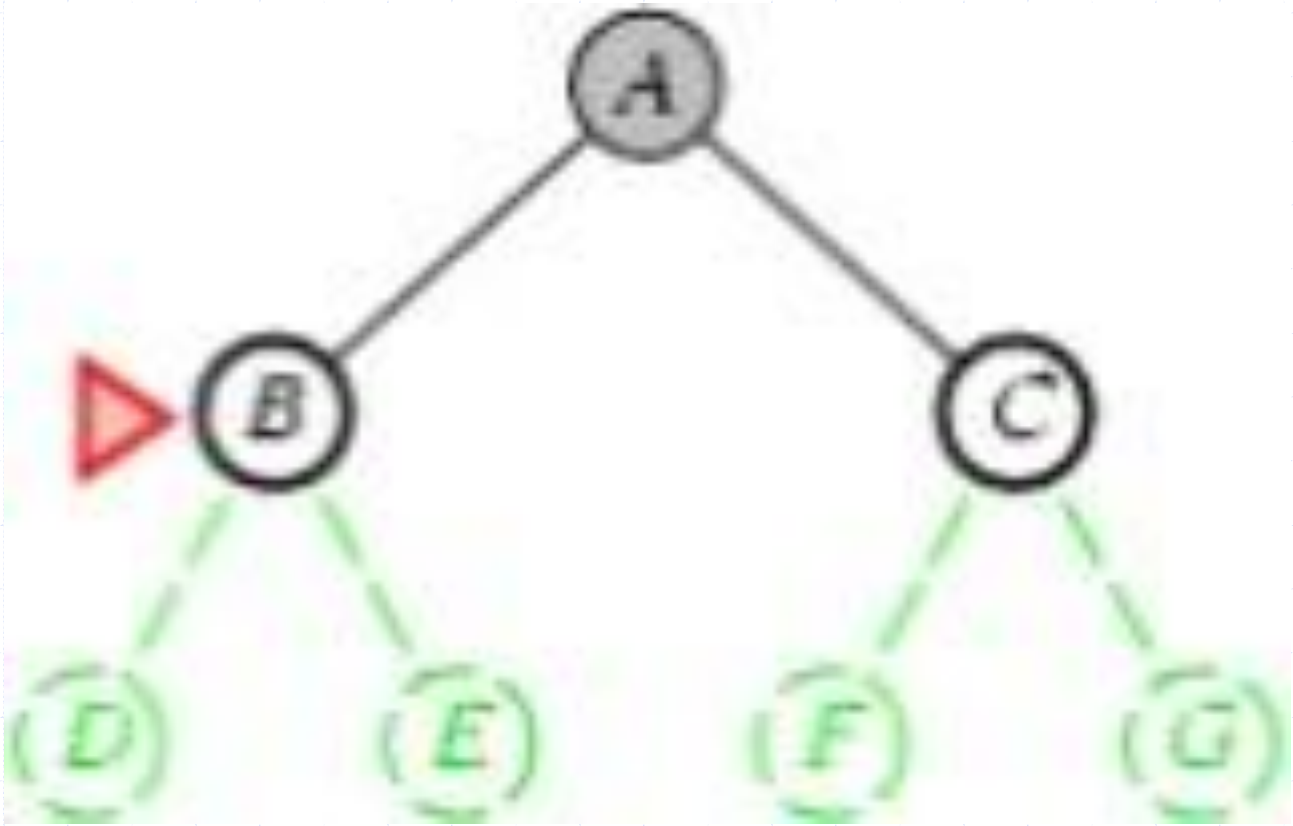
Implementation:

- fringe is a FIFO queue, i.e., new successors go at end



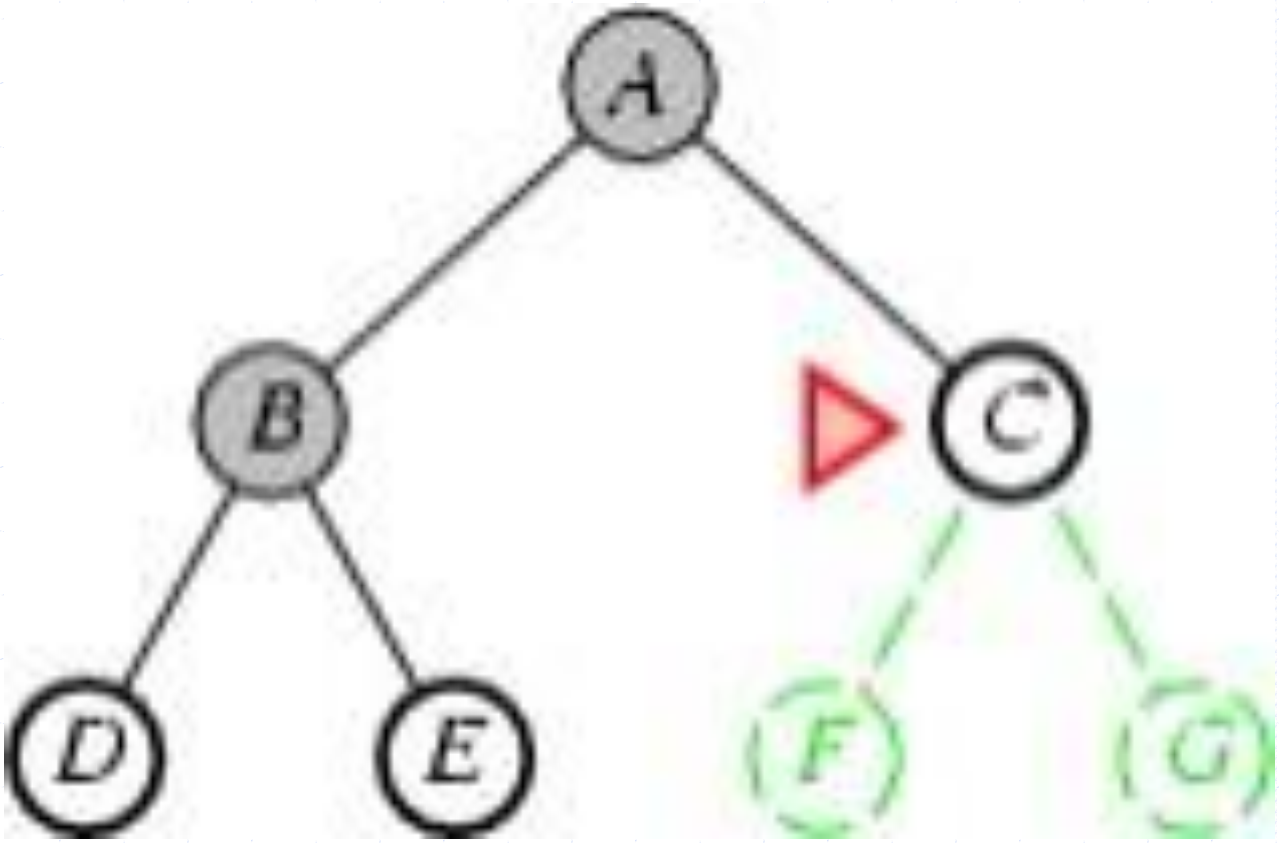
Implementation:

- fringe is a FIFO queue, i.e., new successors go at end



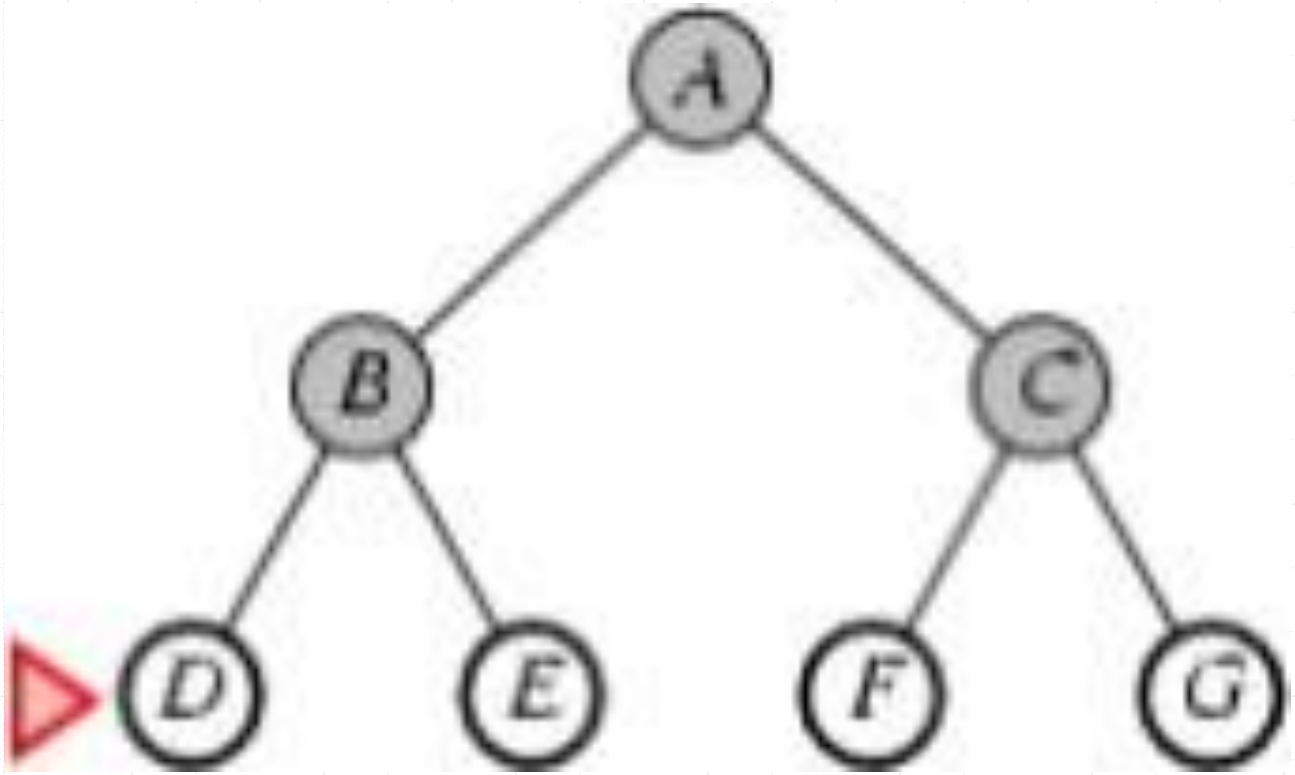
Implementation:

- fringe is a FIFO queue, i.e., new successors go at end



Implementation:

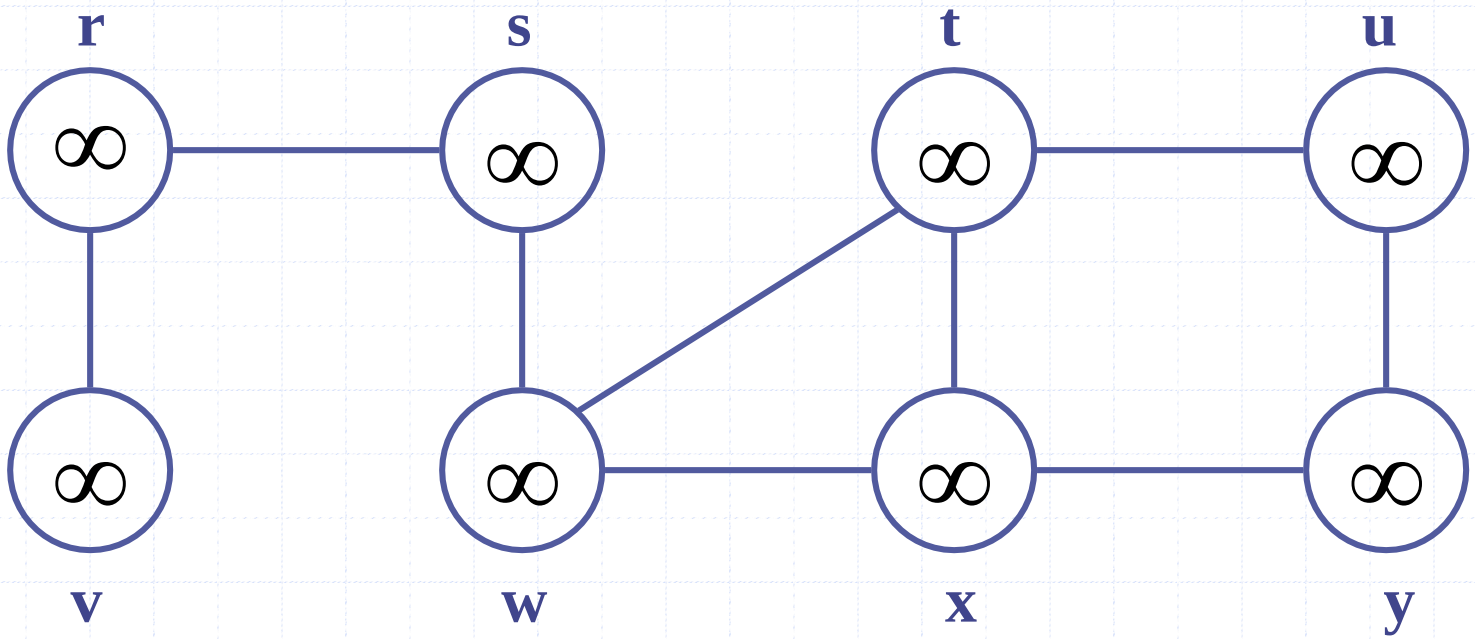
- fringe is a FIFO queue, i.e., new successors go at end



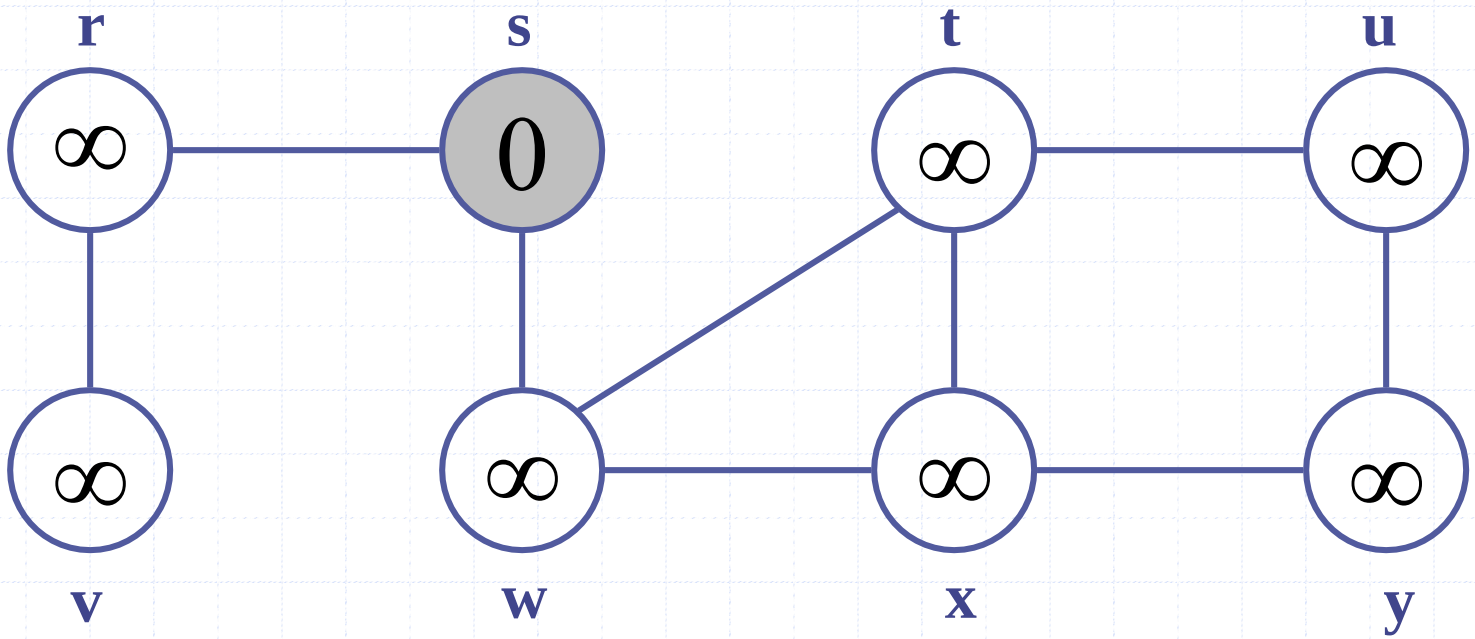
BFS - algorithm

```
BFS(G, s)                          // G is the graph and s is the starting node
 1  for each vertex u ∈ V [G] - {s}
 2       do color[u] ← WHITE       // color of vertex u
 3          d[u] ← ∞               // distance from source s to vertex u
 4          π[u] ← NIL             // predecessor of u
 5 color[s] ← GRAY
 6 d[s] ← 0
 7 π[s] ← NIL
 8 Q ← Ø                           // Q is a FIFO - queue
 9 ENQUEUE(Q, s)
10 while Q ≠ Ø                     // iterates as long as there are gray vertices. Lines 10-18
11       do u ← DEQUEUE(Q)
12          for each v ∈ Adj[u]
13              do if color[v] = WHITE          // discover the undiscovered adjacent vertices
14                    then color[v] ← GRAY      // enqueued whenever painted gray
15                         d[v] ← d[u] + 1
16                         π[v] ← u
17                         ENQUEUE(Q, v)
18          color[u] ← BLACK       // painted black whenever dequeued
```

BFS: Example

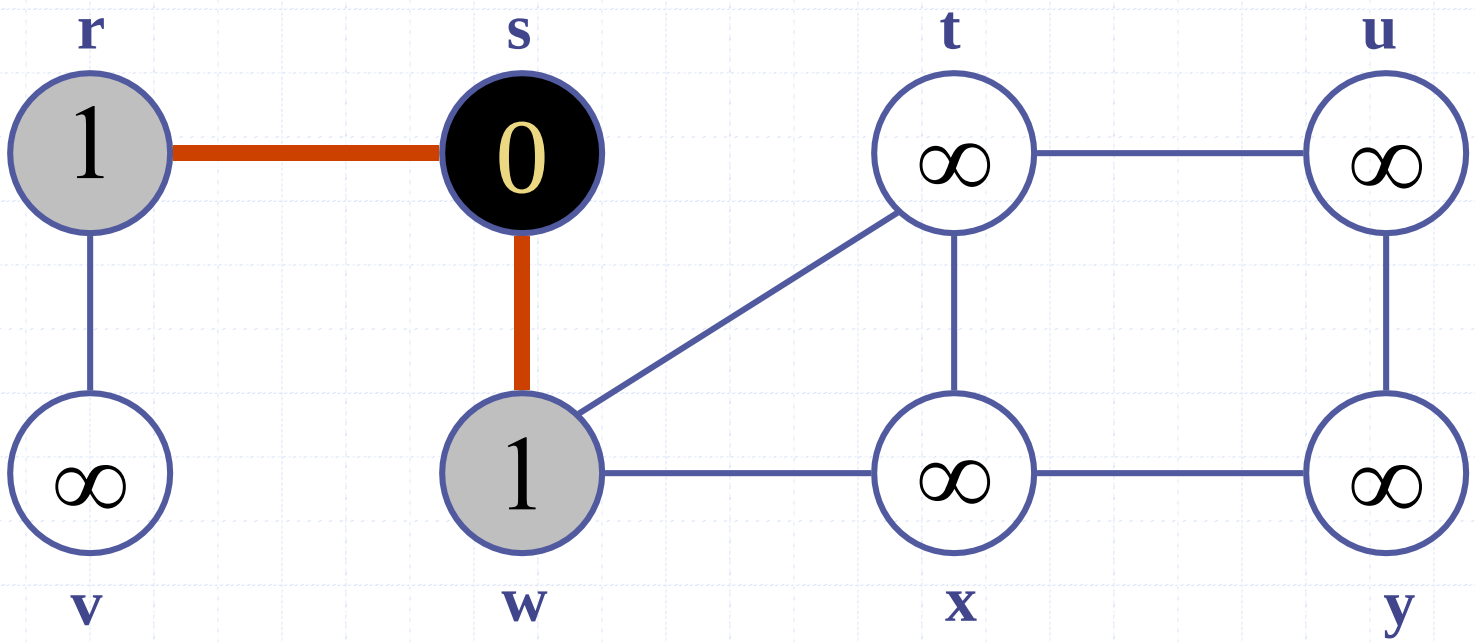


BFS: Example (cont.)



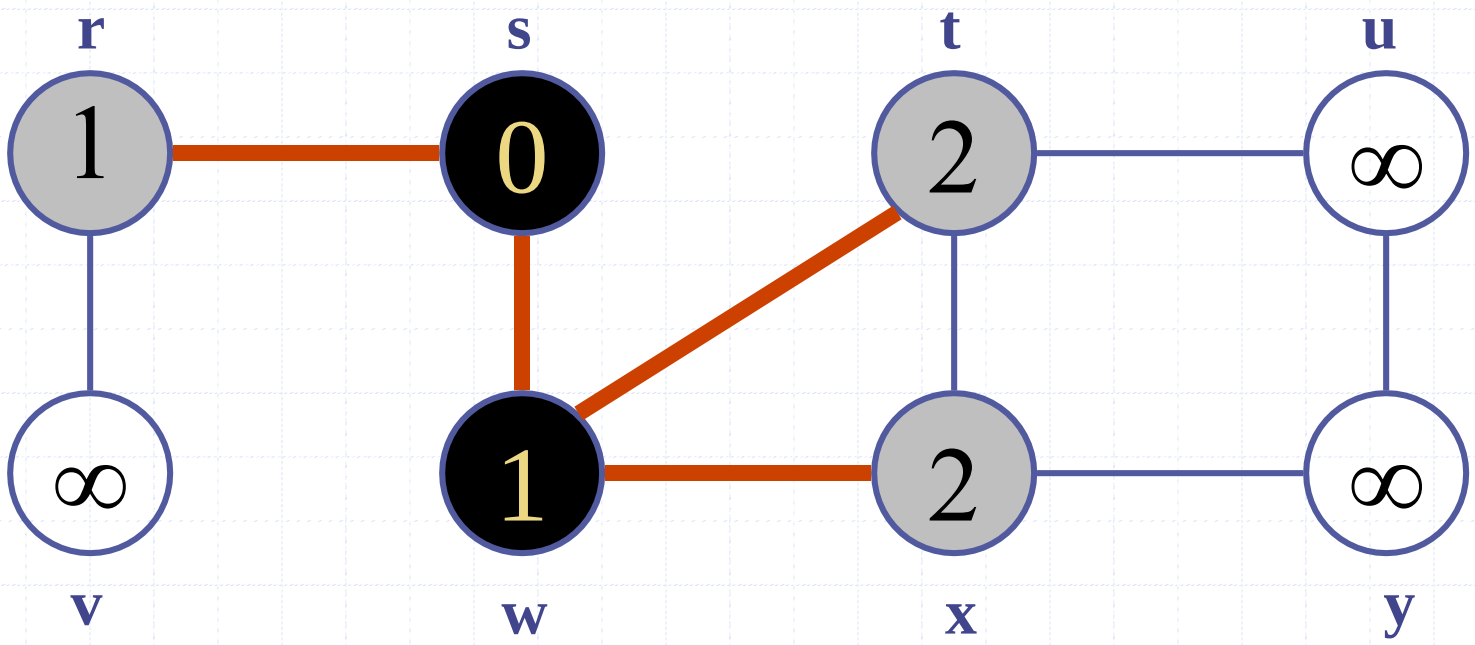
Q: s

BFS: Example (cont.)



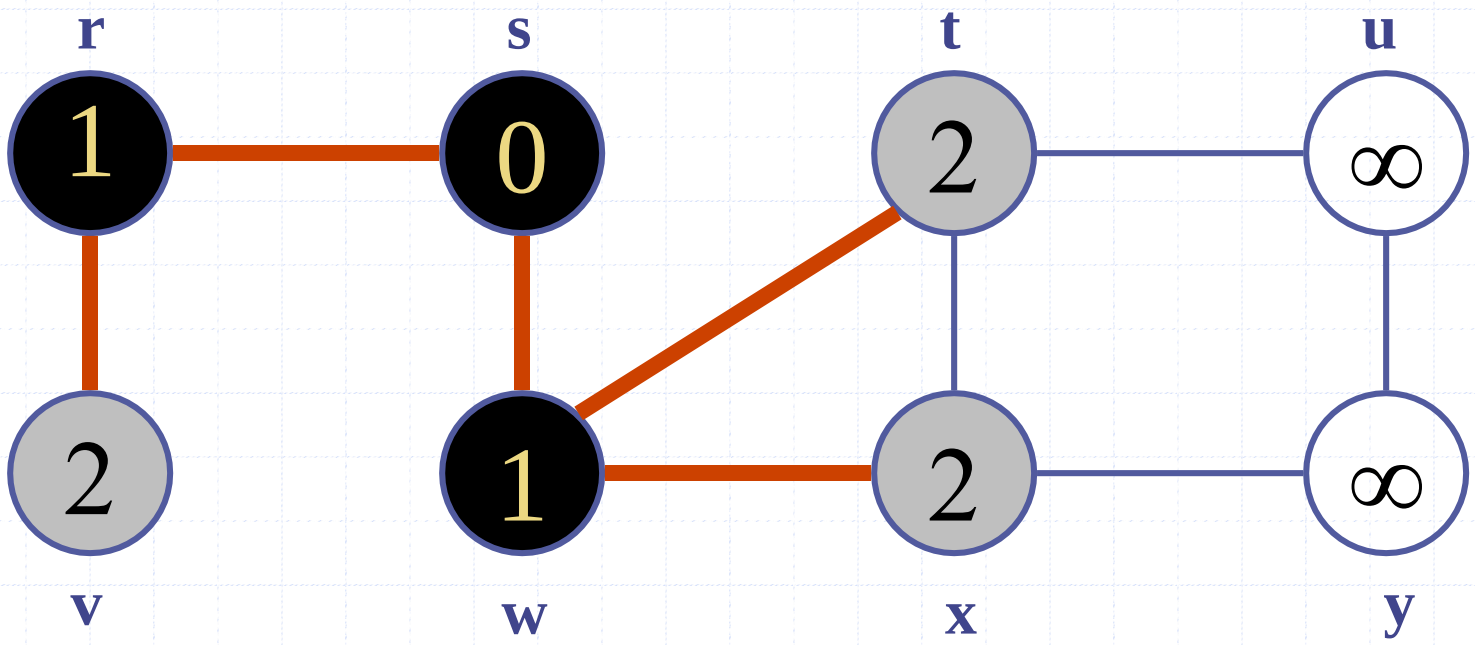
Q: | w | r |

BFS: Example (cont.)



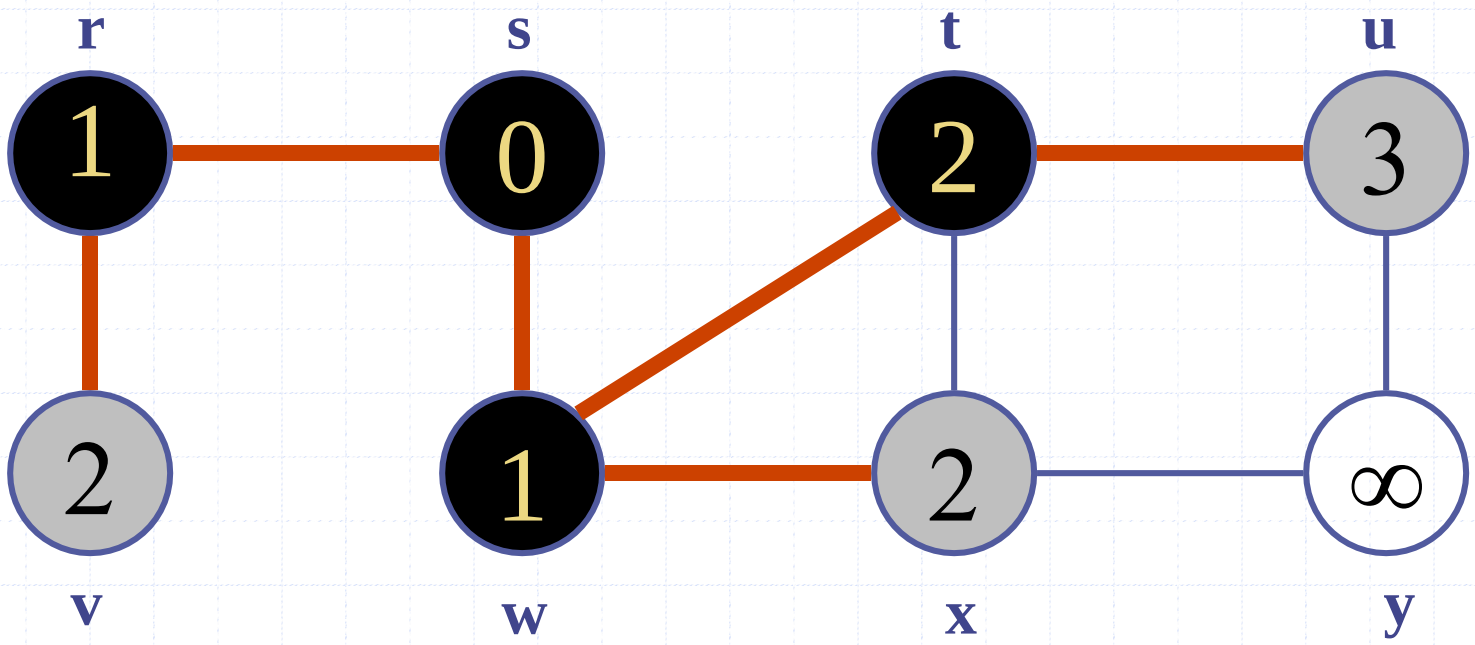
Q: | r | t | x |

BFS: Example (cont.)



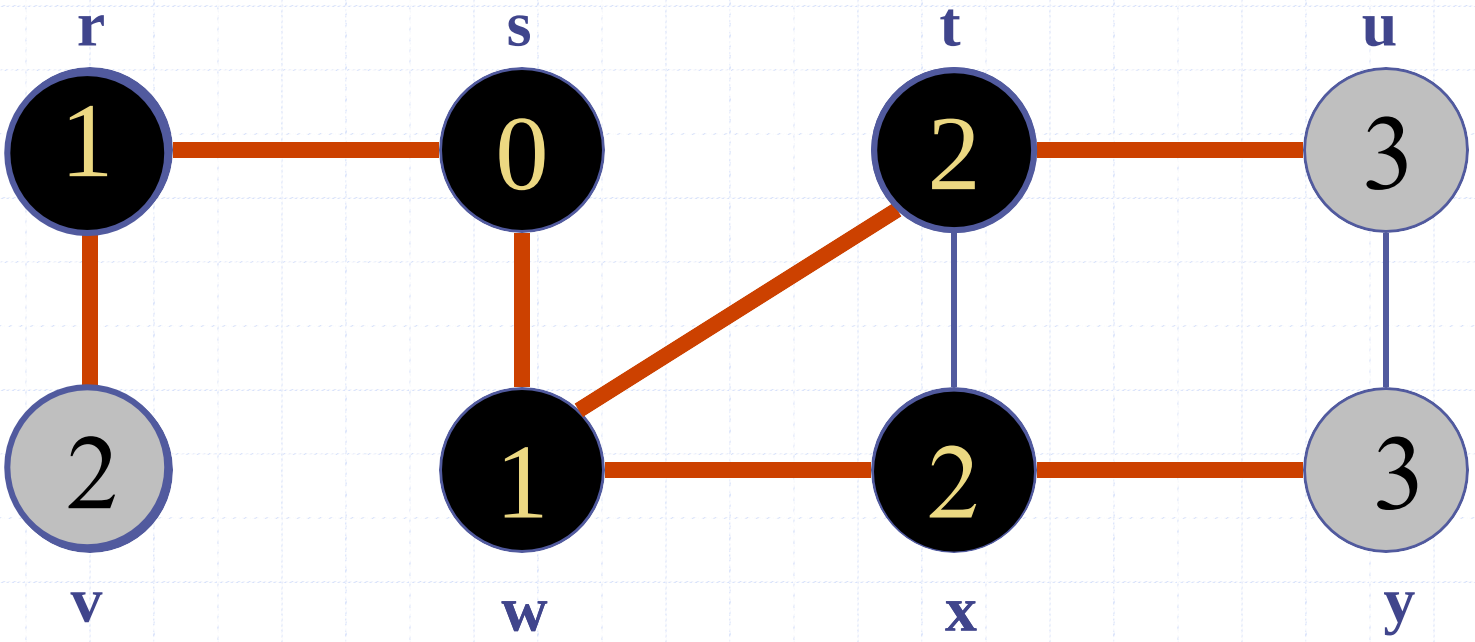
Q: | t | x | v |

BFS: Example (cont.)



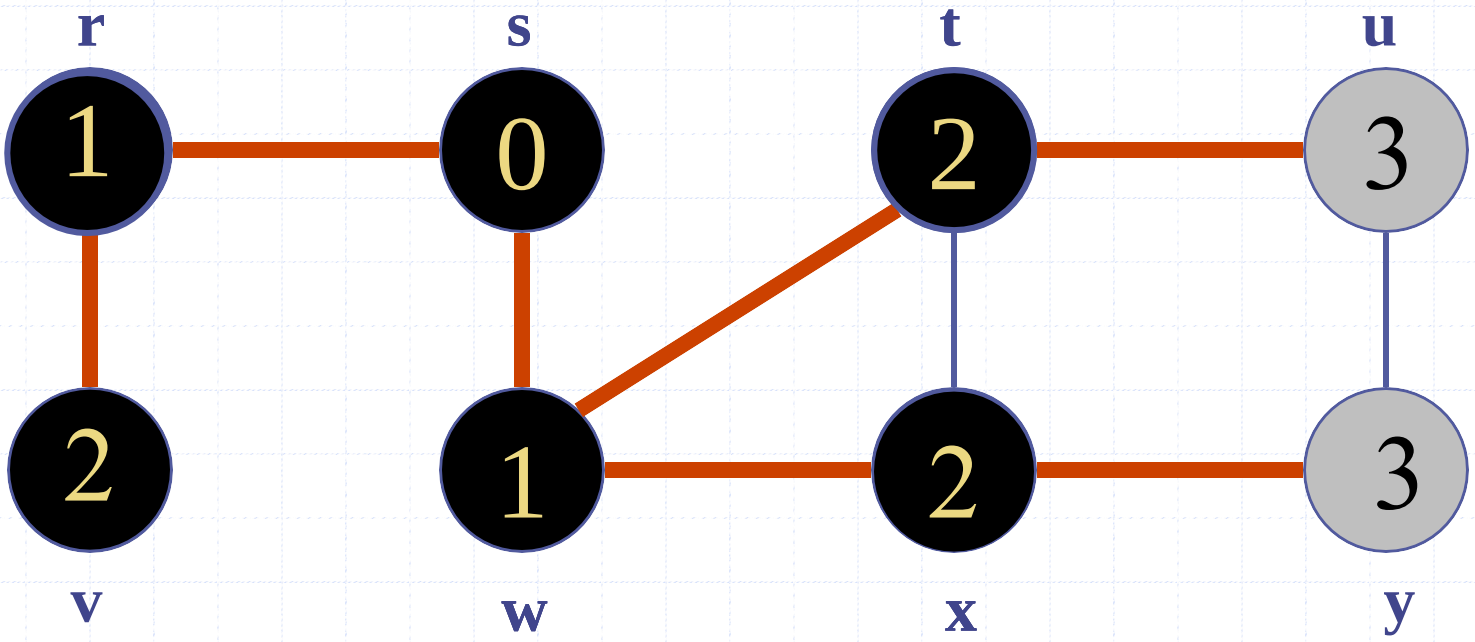
Q: | x | v | u |

BFS: Example (cont.)



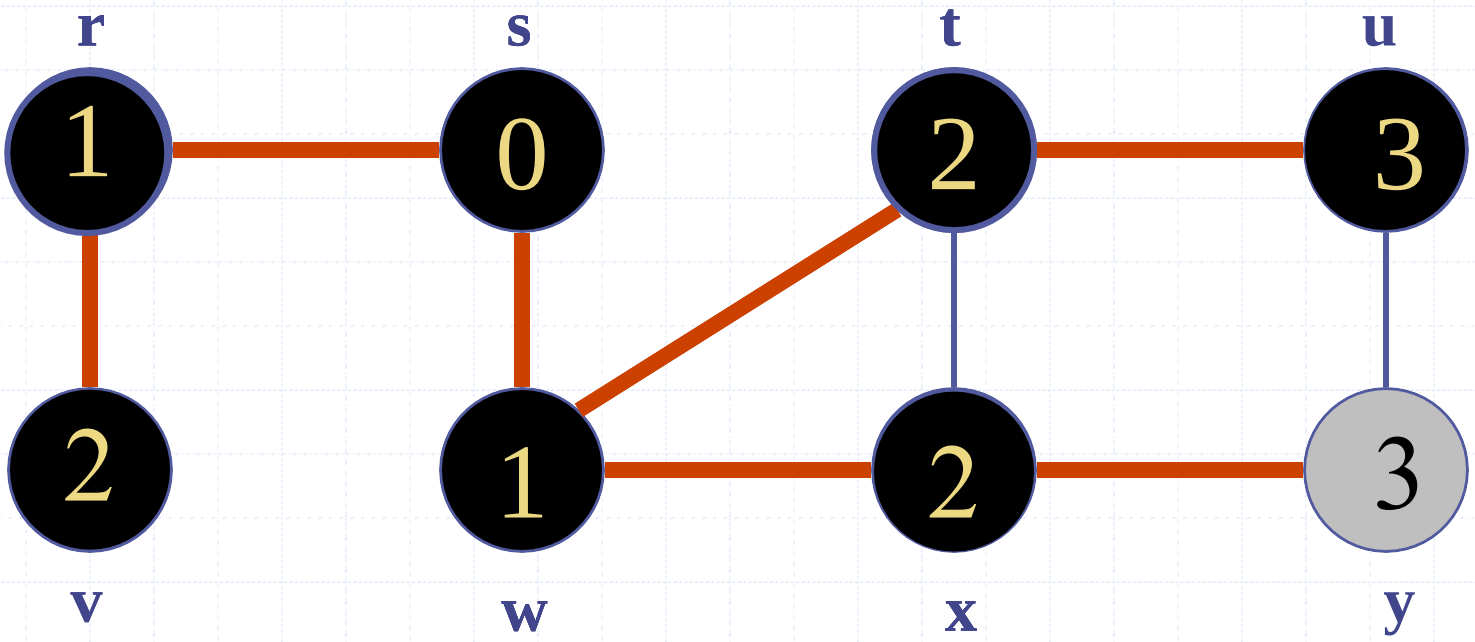
Q: | v | u | y |

BFS: Example (cont.)



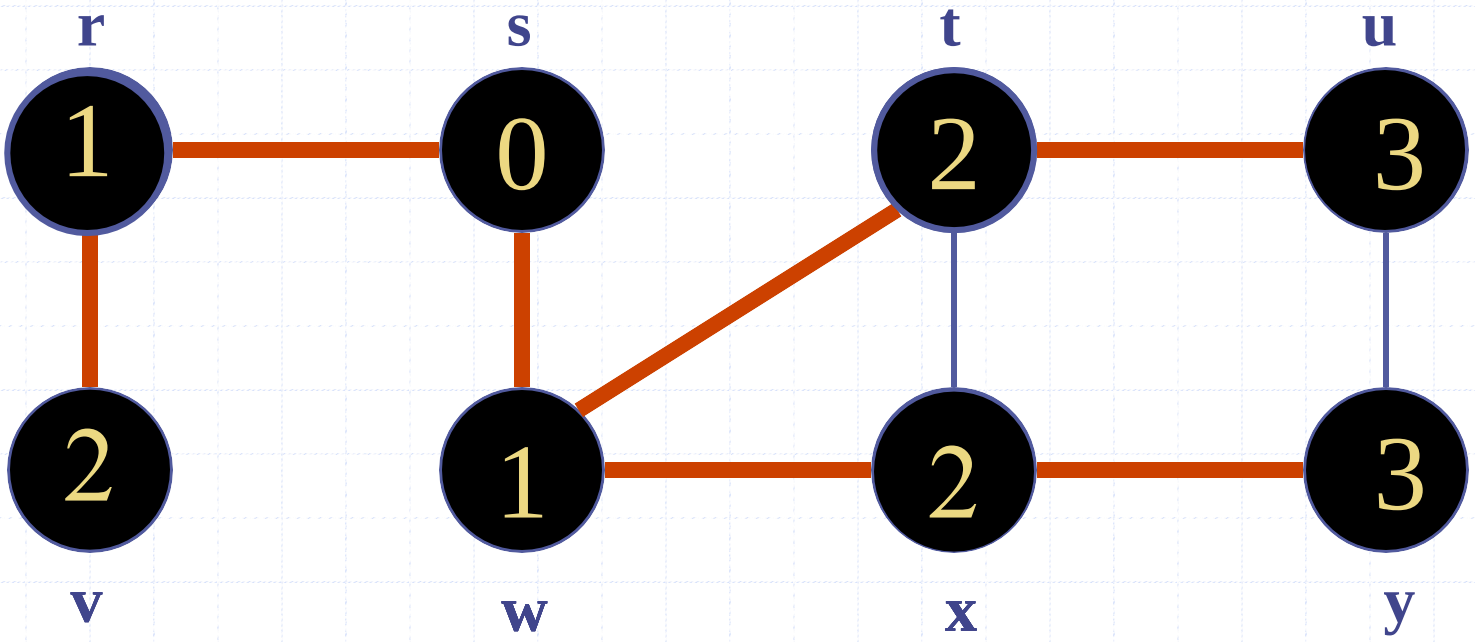
Q: | u | y |

BFS: Example (cont.)



Q: y

BFS: Example (cont.)



Q: Ø

Analysis of BFS

- The operations of enqueuing and dequeuing take $O(1)$ time. So the total time devoted to queue operations is $O(V)$.
- Because the adjacency list of each vertex is scanned only when the vertex is dequeued, each adjacency list is scanned at most once.
- Since the sum of the lengths of all the adjacency lists is $\Theta(E)$, the total time spent in scanning adjacency lists is $O(E)$.
- The overhead for initialization is $O(V)$, and thus the total running time of BFS is $O(V + E)$.
- Thus breadth-first search runs in time linear in the size of the adjacency-list representation of G.

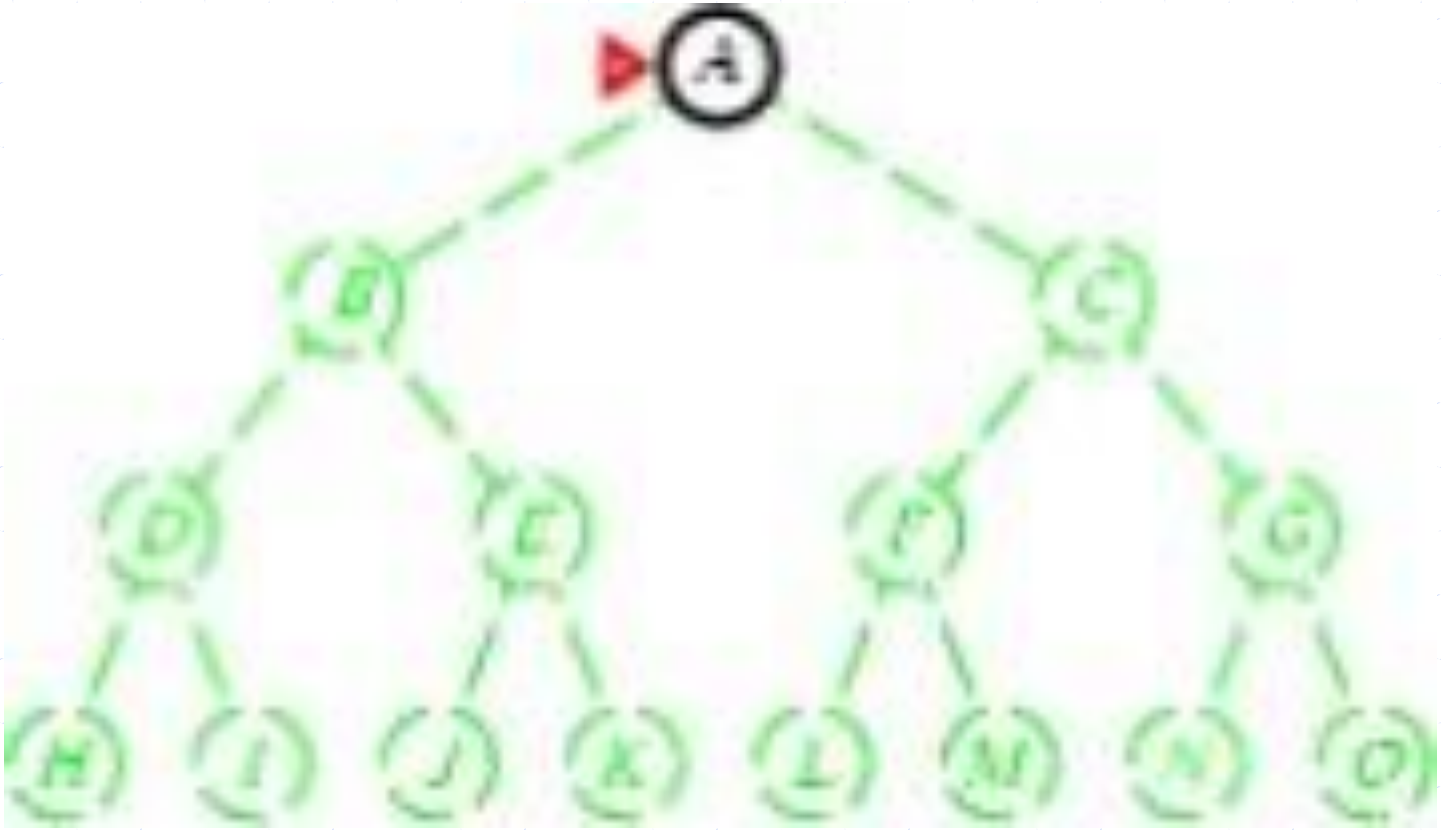
BFS: Properties

- BFS calculates the shortest-path distance to the source node
  - Shortest-path distance $\delta(s,v)$ = minimum number of edges from s to v, or ∞ if v not reachable from s.
- BFS builds breadth-first tree, in which paths to root represent shortest paths in G.
  - Thus can use BFS to calculate shortest path from one vertex to another in O(V+E) time.

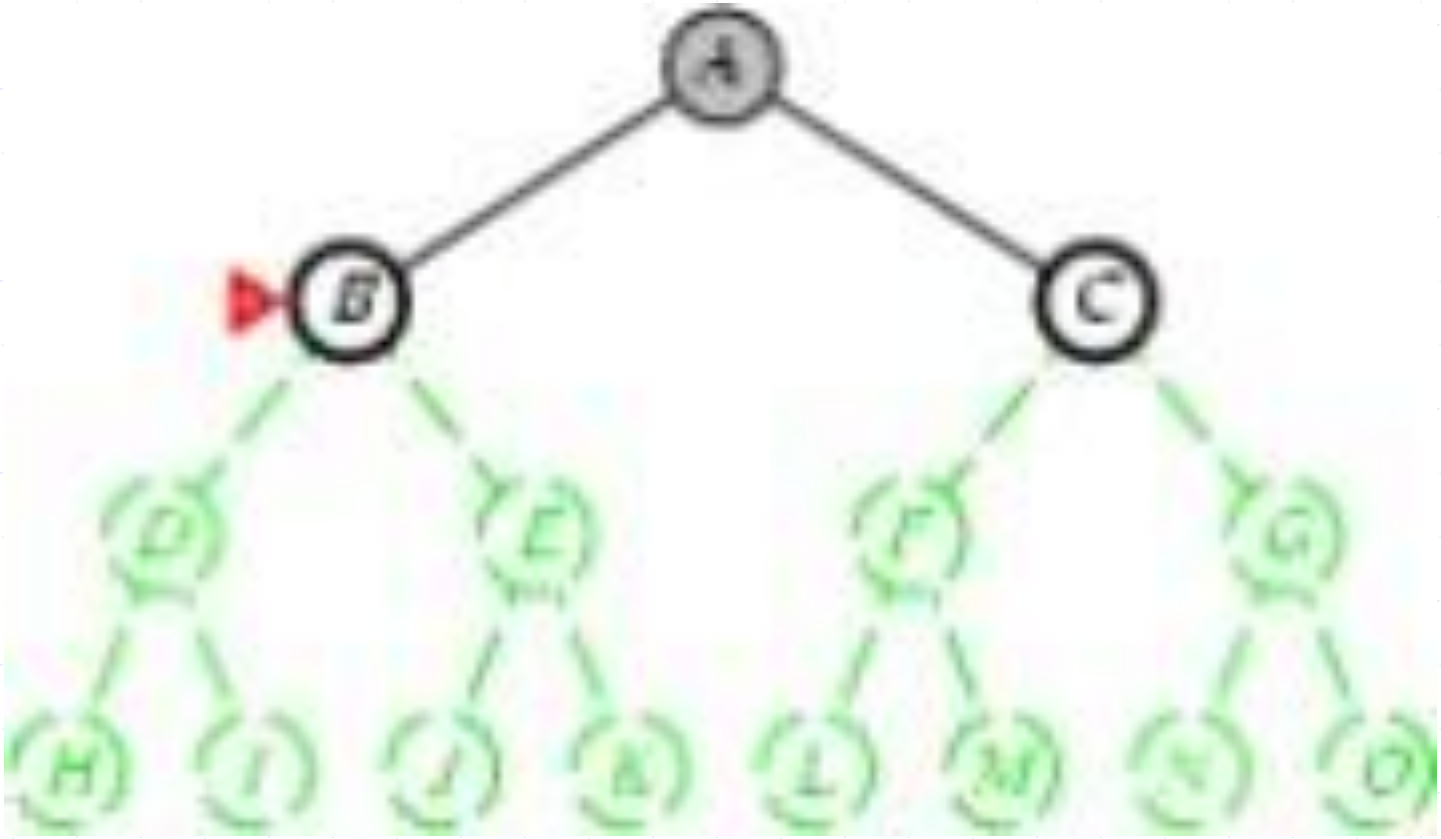
Depth first search (DFS)

- Depth-first search is a strategy for exploring a graph
  - Explore "deeper" in the graph whenever possible.
  - Edges are explored out of the most recently discovered vertex v that still has unexplored edges.
  - When all of v's edges have been explored, backtrack to the vertex from which v was discovered.

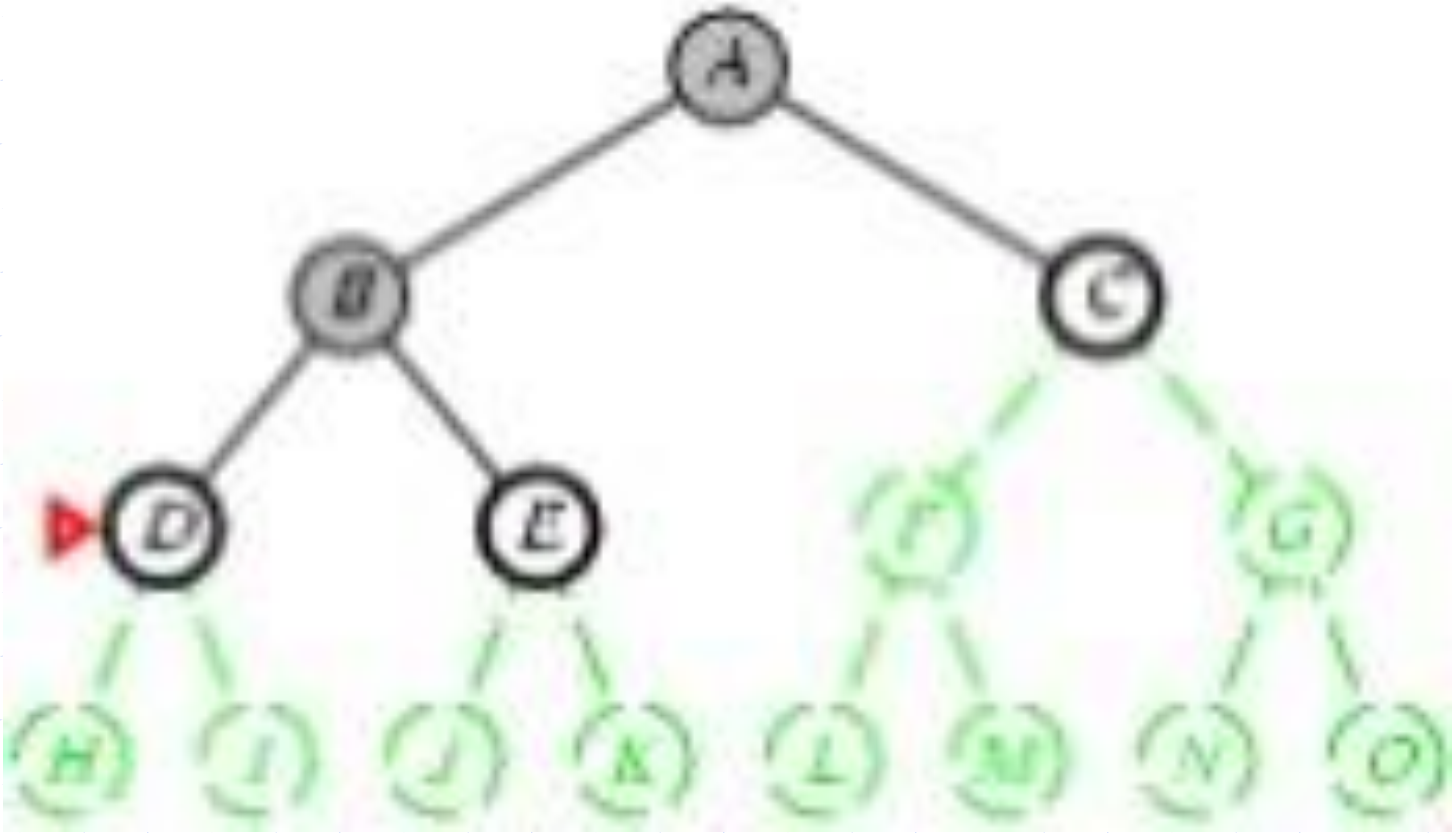
- Expand deepest unexpanded node.
- Implementation:
  fringe = LIFO queue, i.e., put successors at front



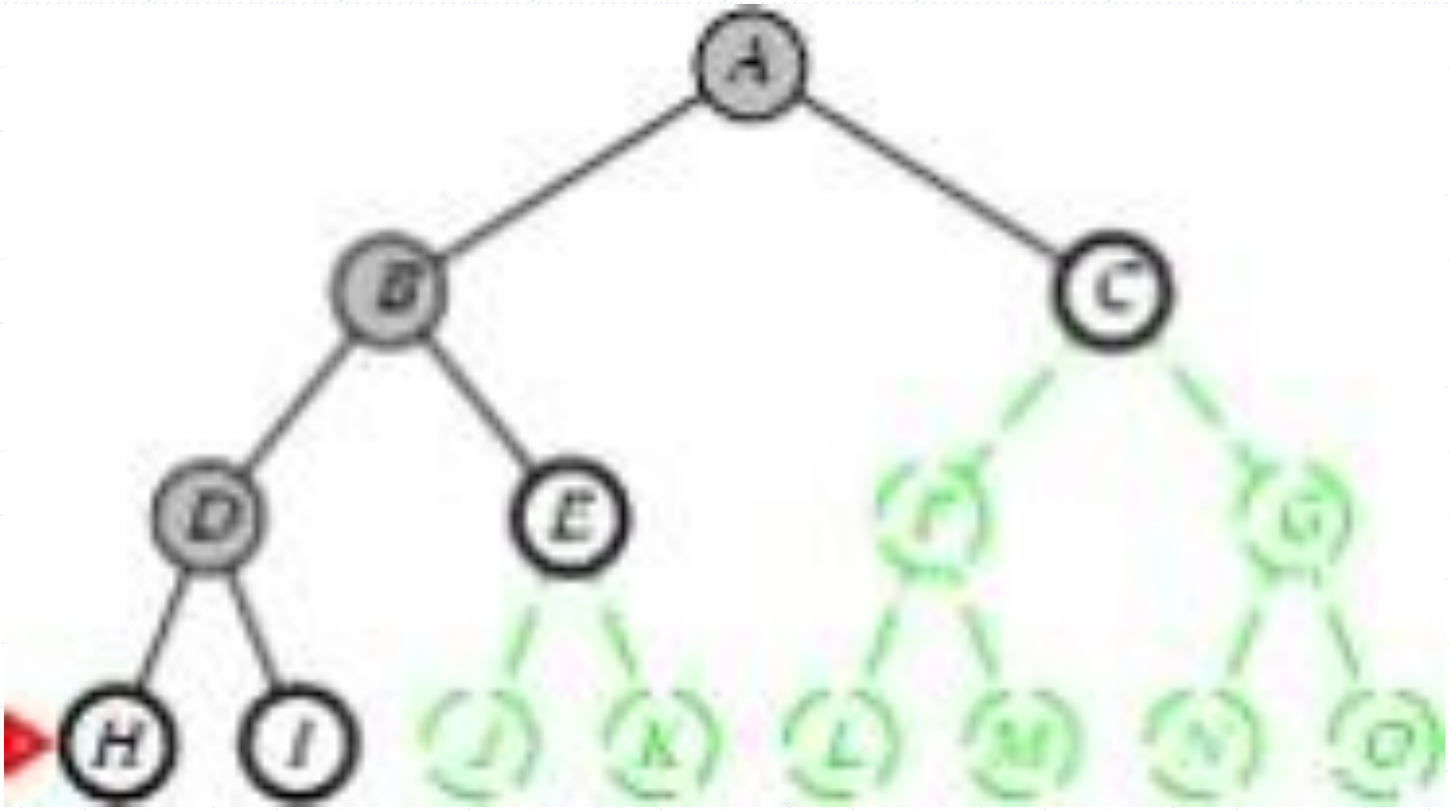
- Expand deepest unexpanded node.
- Implementation:
  fringe = LIFO queue, i.e., put successors at front



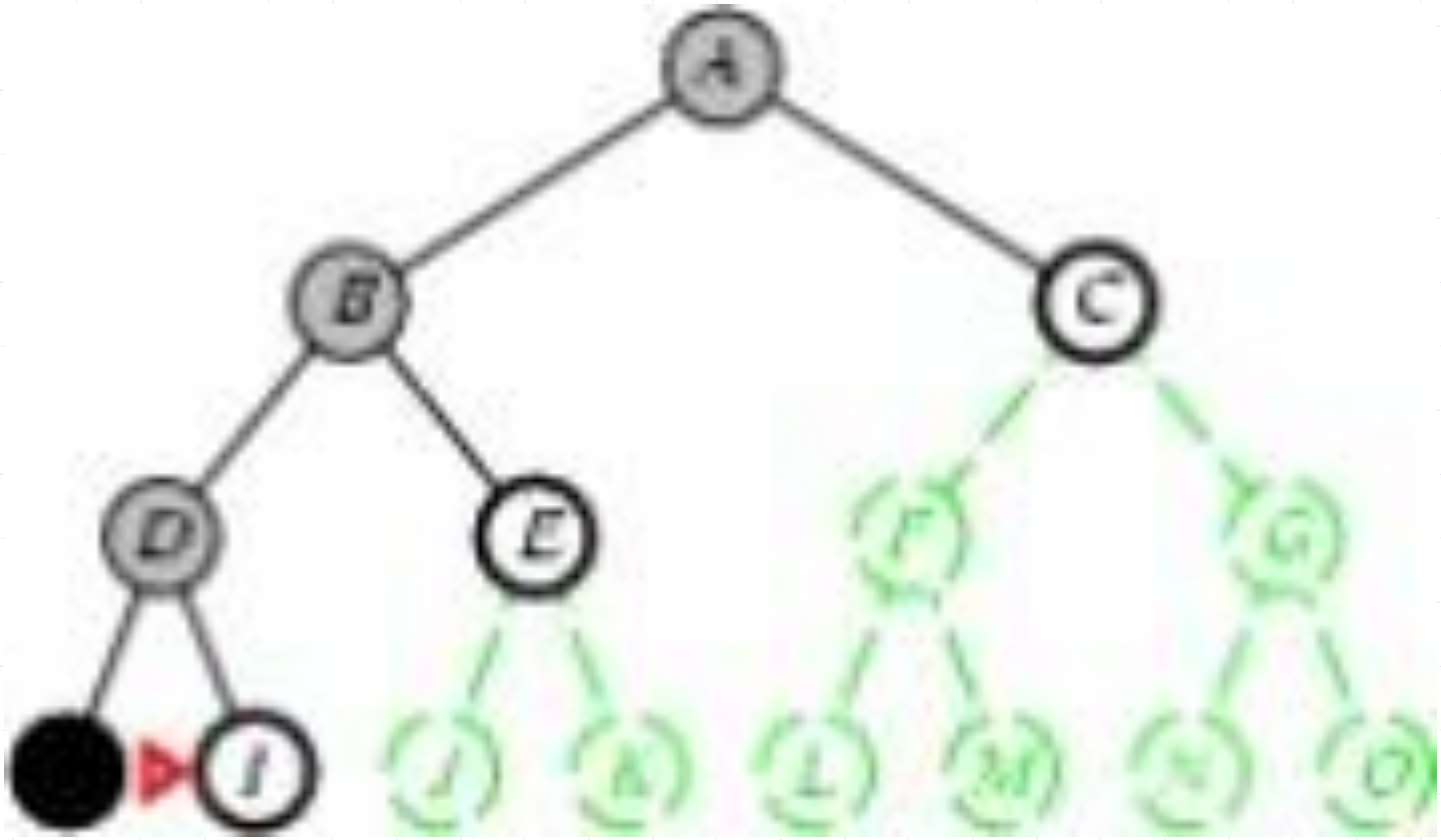
- Expand deepest unexpanded node.
- Implementation:
  fringe = LIFO queue, i.e., put successors at front



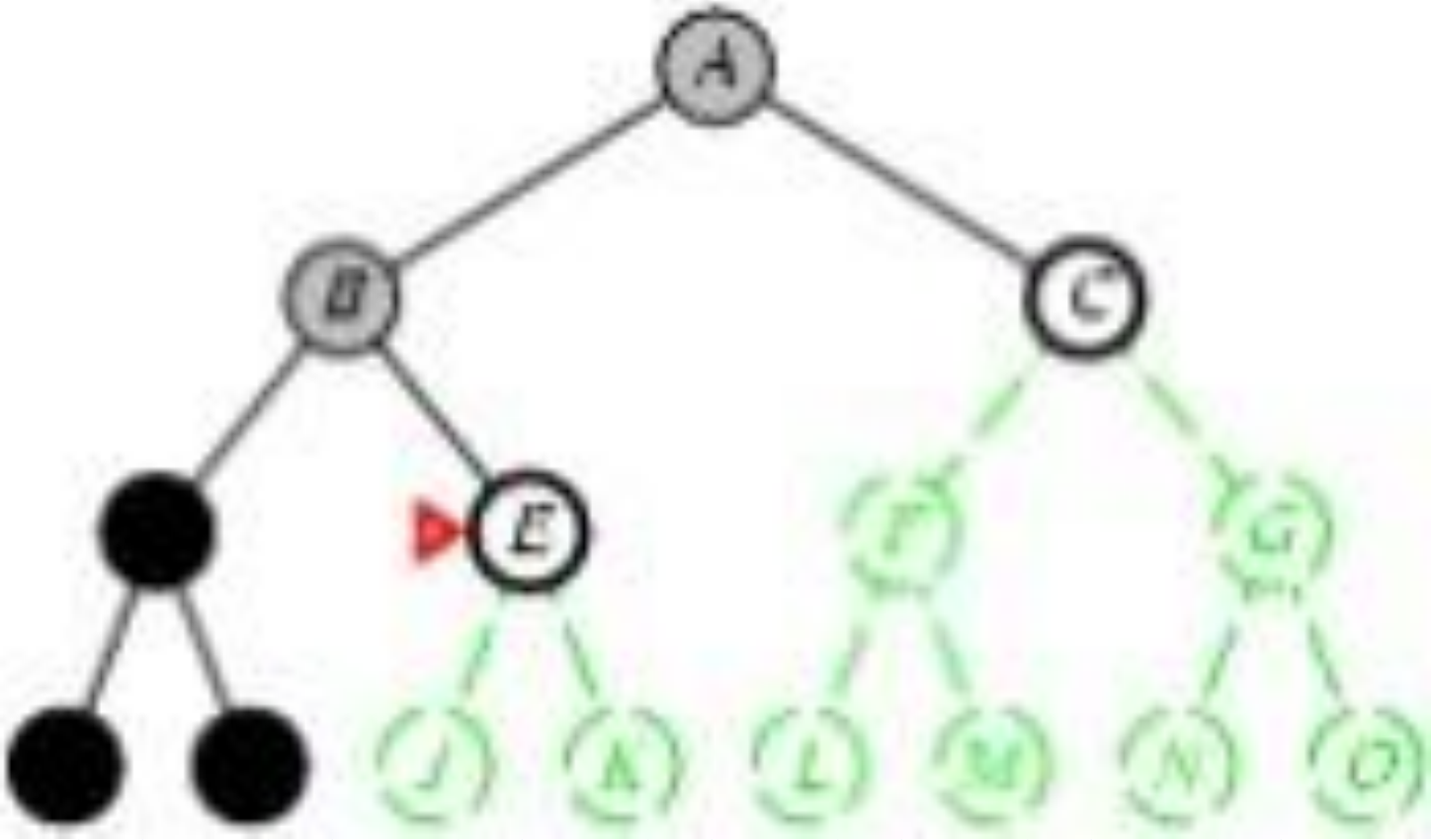
- Expand deepest unexpanded node.
- Implementation:
  fringe = LIFO queue, i.e., put successors at front



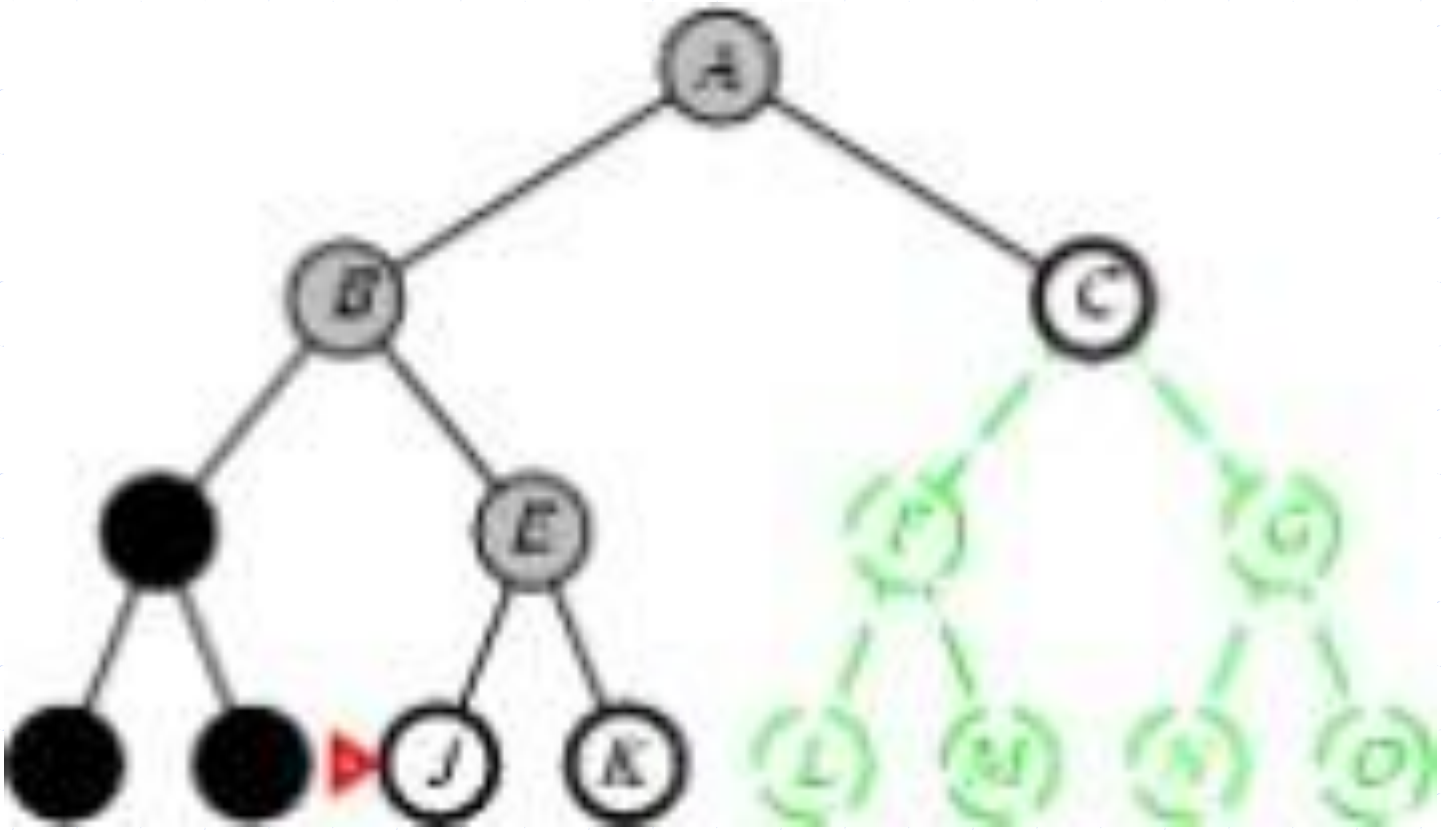
- Expand deepest unexpanded node.
- Implementation:
  fringe = LIFO queue, i.e., put successors at front



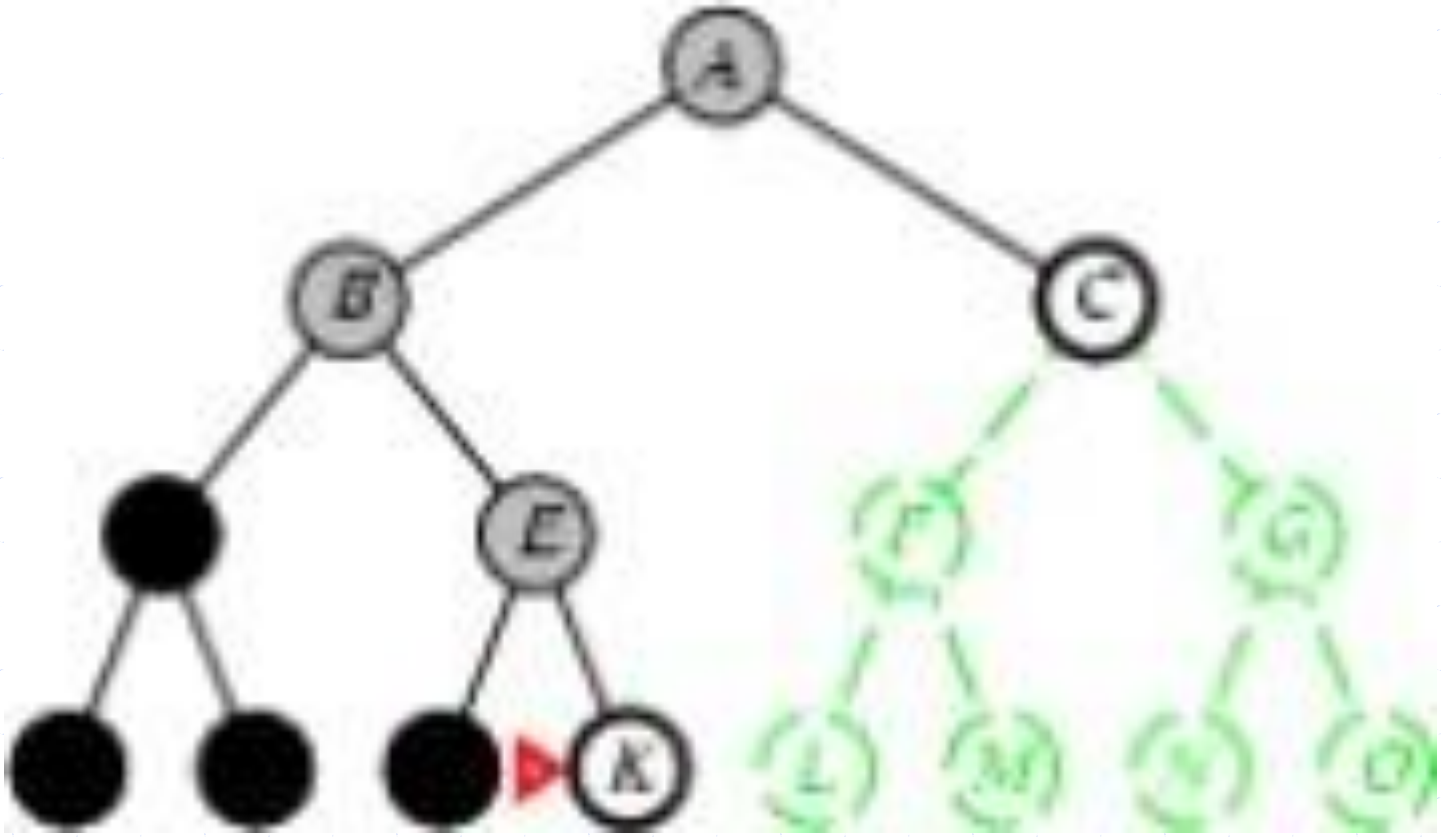
- Expand deepest unexpanded node.
- Implementation:
  fringe = LIFO queue, i.e., put successors at front



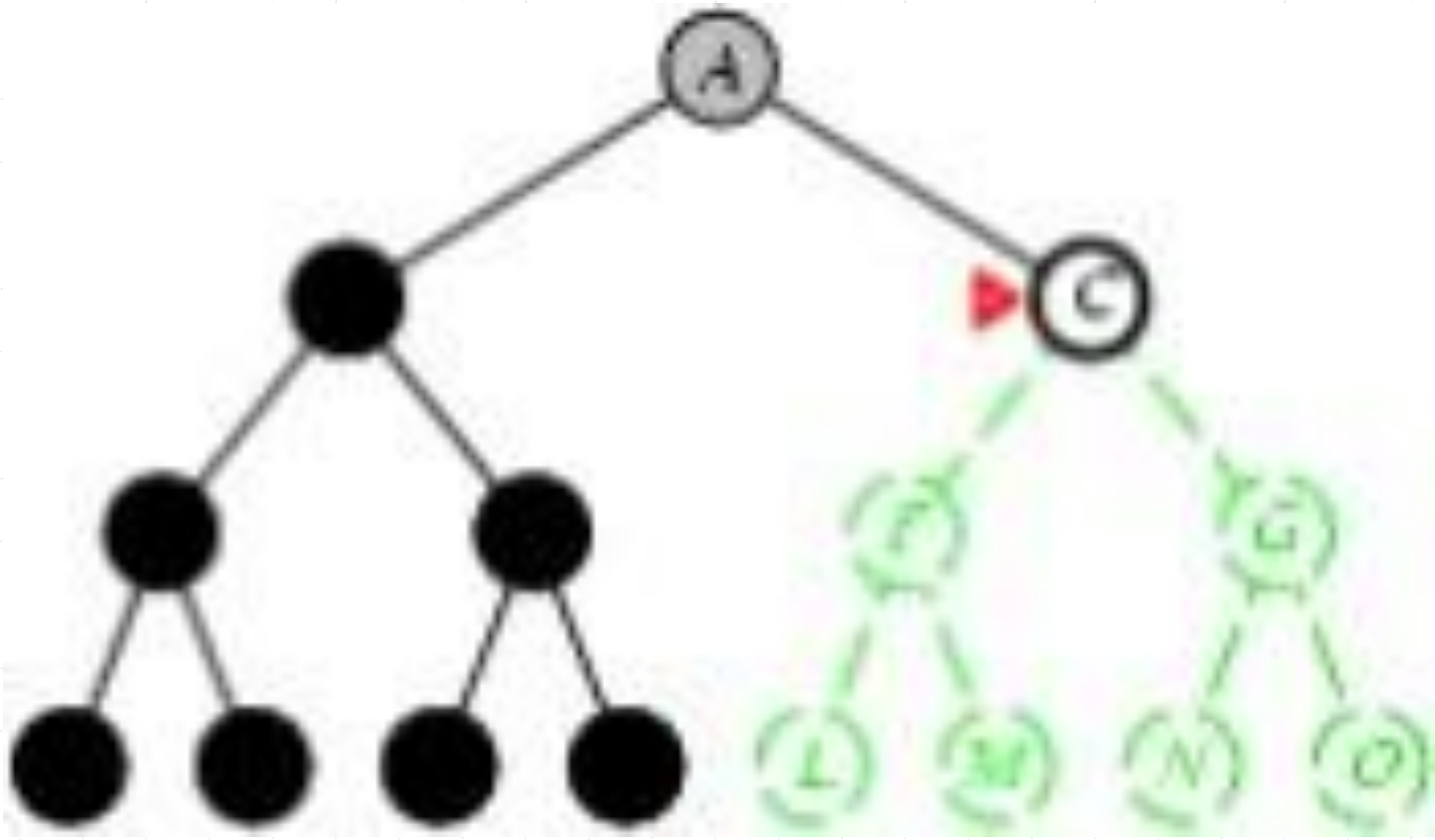
- Expand deepest unexpanded node.
- Implementation:
  fringe = LIFO queue, i.e., put successors at front



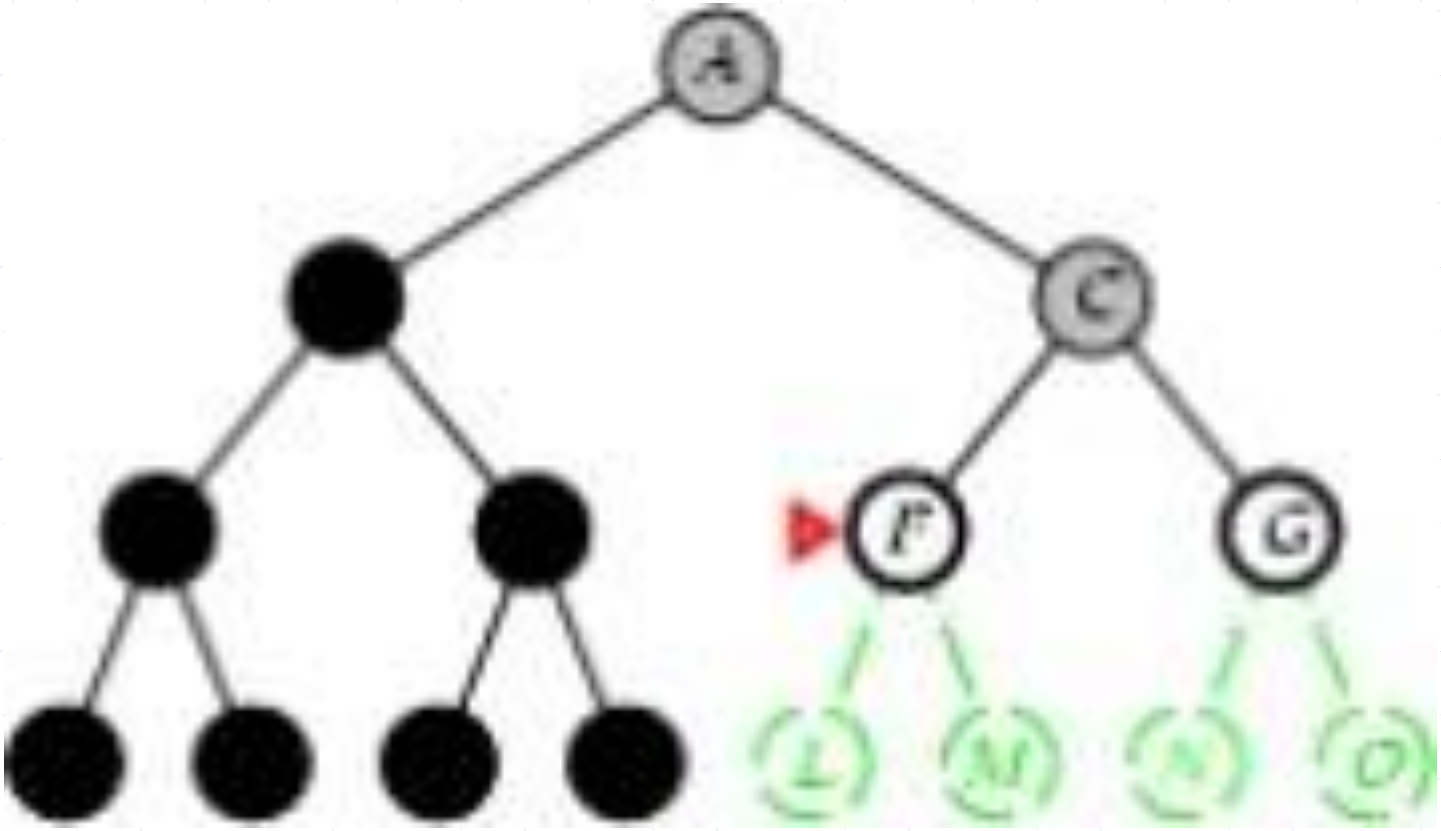
- Expand deepest unexpanded node.
- Implementation:
  fringe = LIFO queue, i.e., put successors at front



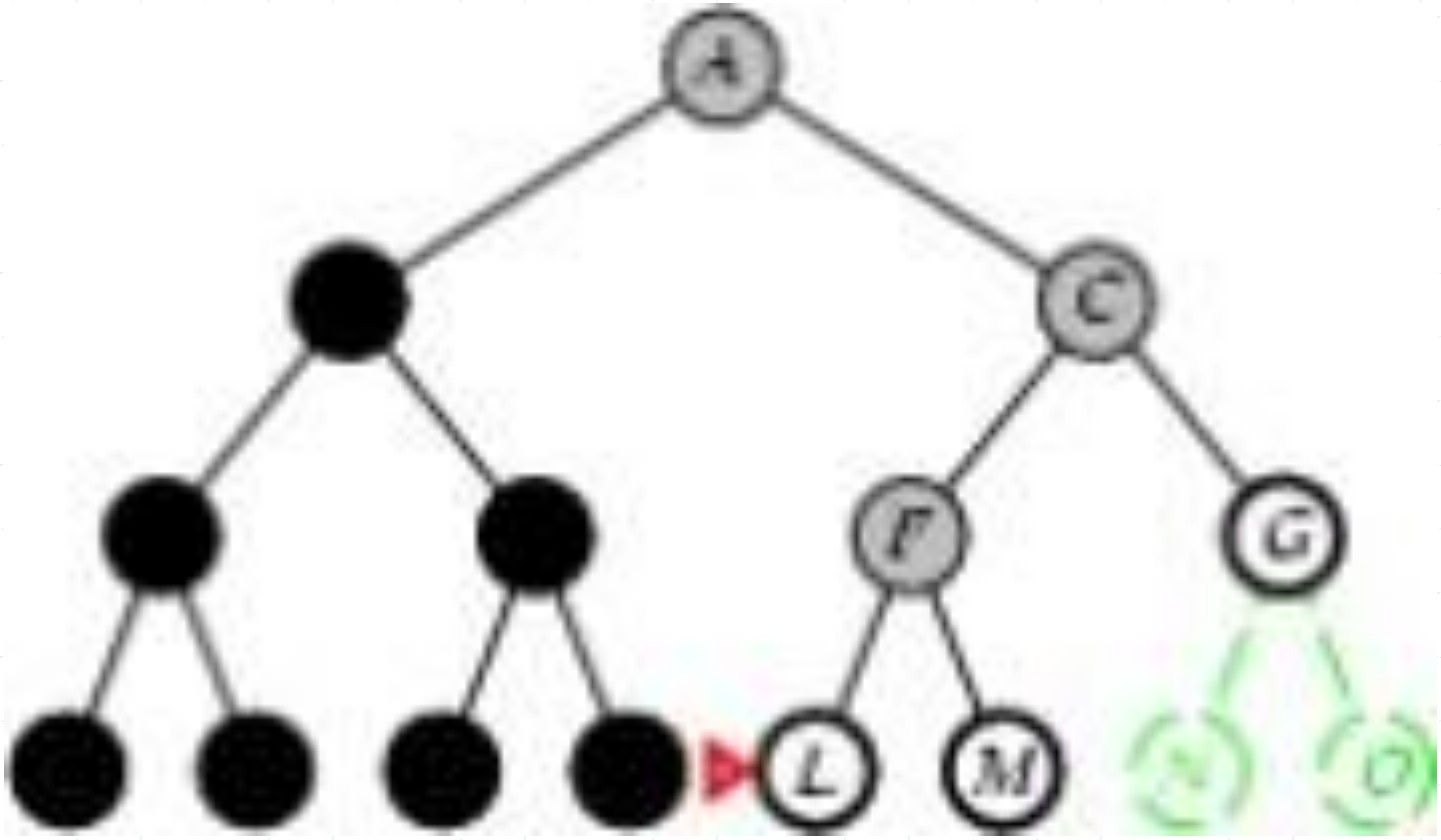
- Expand deepest unexpanded node.
- Implementation:
  fringe = LIFO queue, i.e., put successors at front



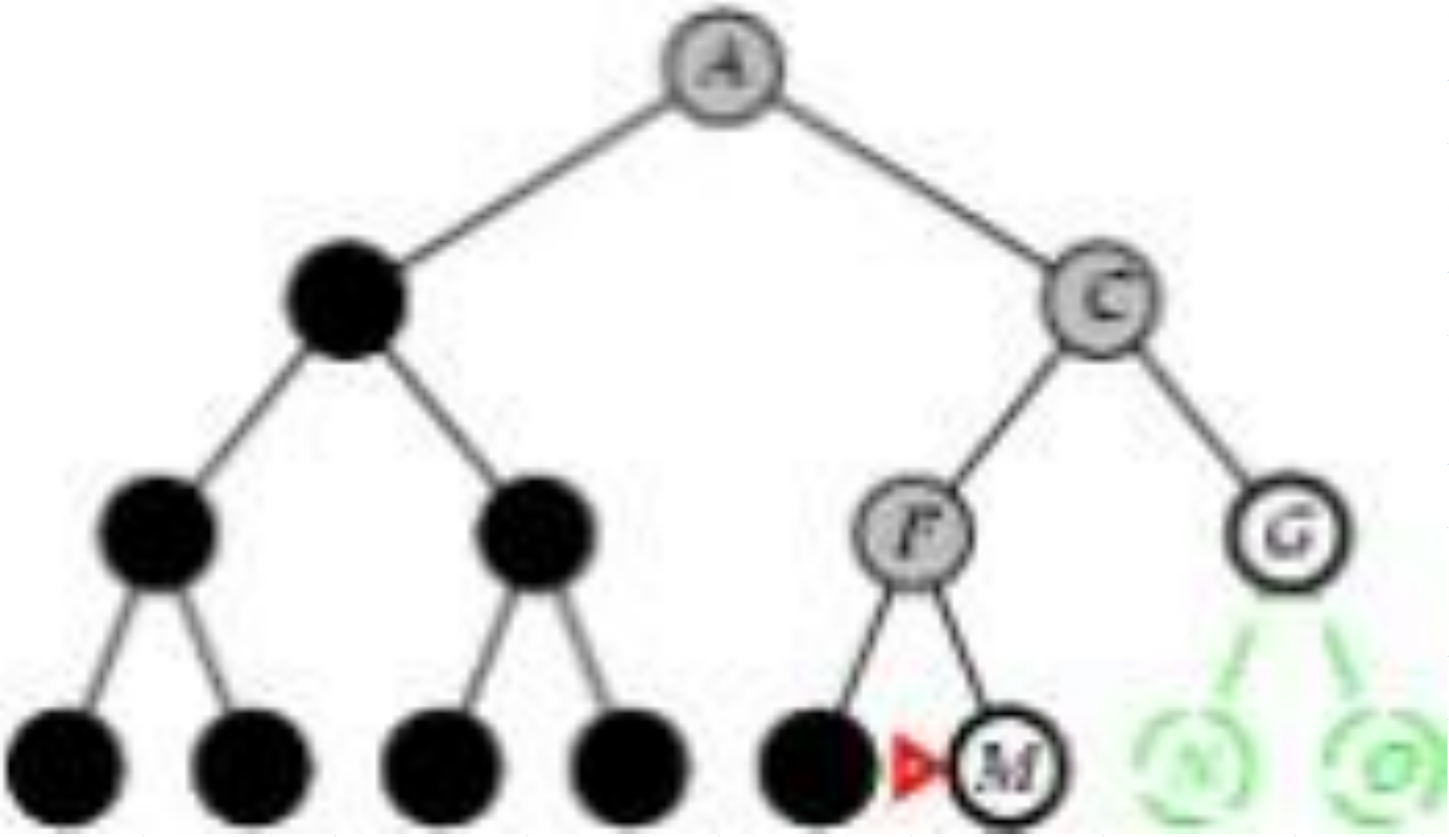
- Expand deepest unexpanded node.
- Implementation:
  fringe = LIFO queue, i.e., put successors at front



- Expand deepest unexpanded node.
- Implementation:
  fringe = LIFO queue, i.e., put successors at front



- Expand deepest unexpanded node.
- Implementation:
  fringe = LIFO queue, i.e., put successors at front



Depth first search (cont.)

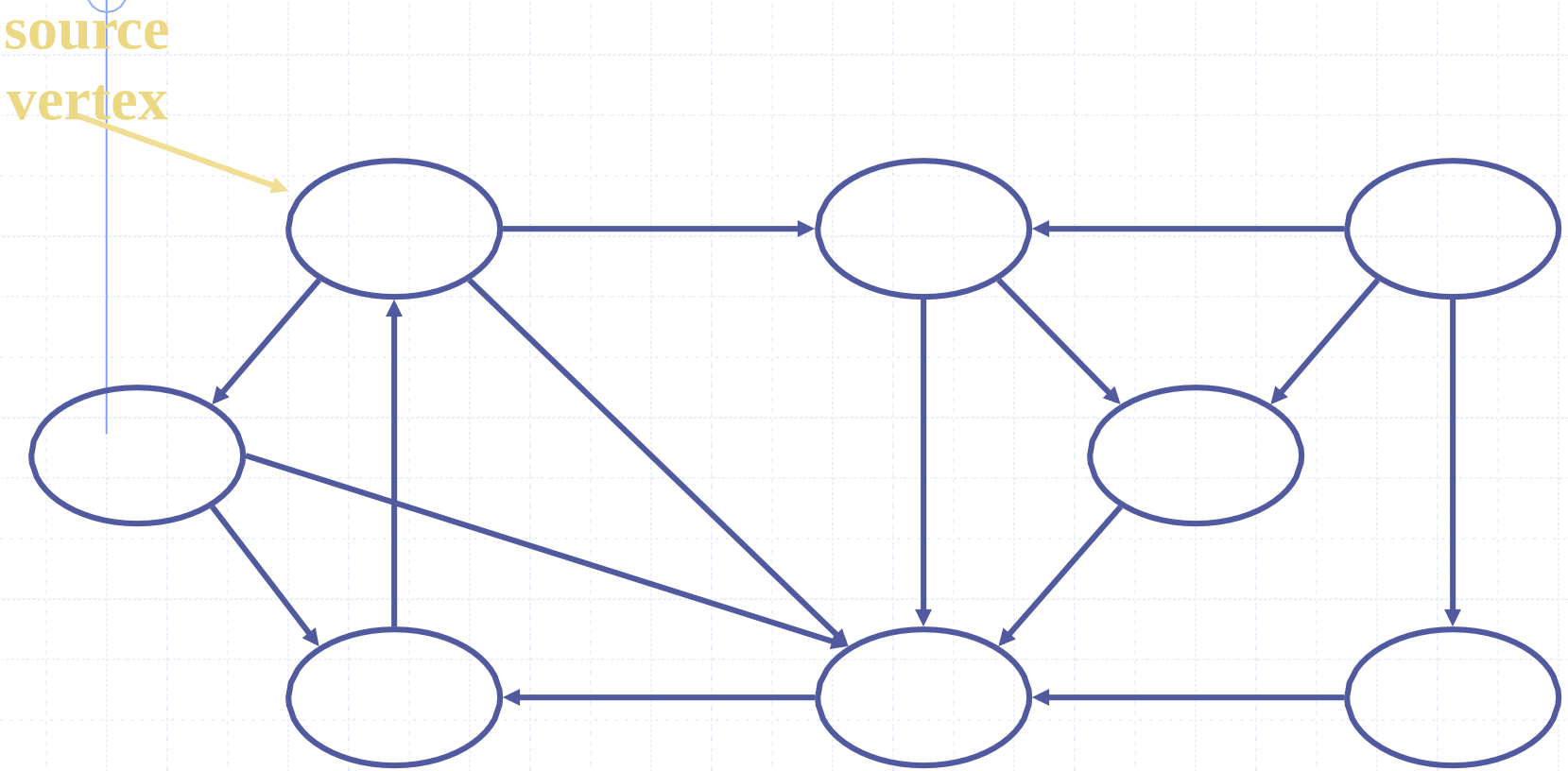
- It searches deeper the graph when possible.
- Starts at the selected node and explores as far as possible along each branch before backtracking.
- Vertices go through white, gray and black stages of color.
  - White – initially
  - Gray – when discovered first
  - Black – when finished i.e. the adjacency list of the vertex is completely examined.
- Also records timestamps for each vertex
  - d[v] when the vertex is first discovered
  - f[v] when the vertex is finished

DFS algorithm (cont.)

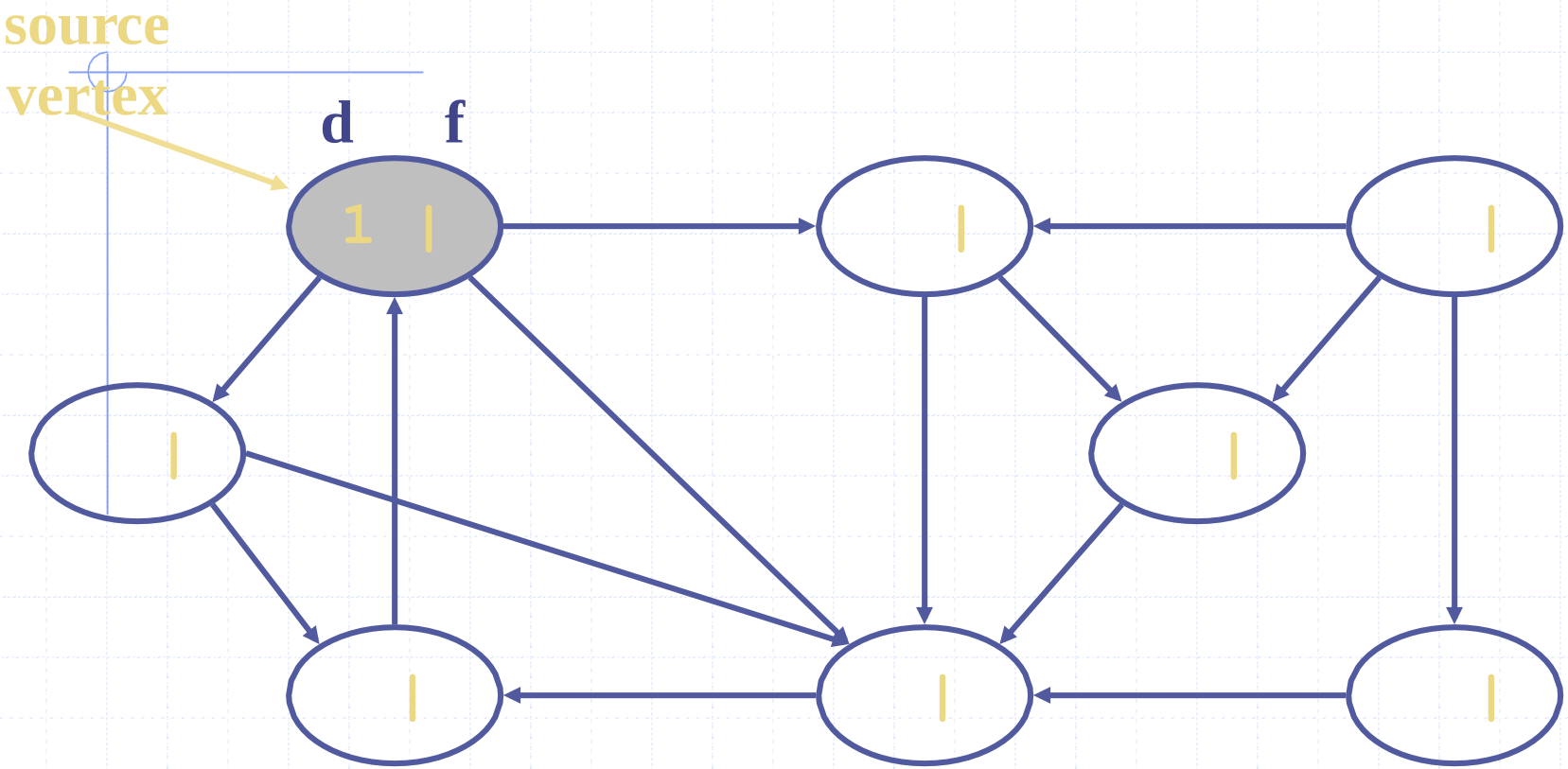
```
DFS(G)
1  for each vertex u ∈ V [G]
2       do color[u] ← WHITE          // color all vertices white, set their parents NIL
3          π[u] ← NIL
4  time ← 0                         // zero out time
5  for each vertex u ∈ V [G]        // call only for unexplored vertices
6       do if color[u] = WHITE                  // this may result in multiple sources
7             then DFS-VISIT(u)

DFS-VISIT(u)
1  color[u] ← GRAY      ▹White vertex u has just been discovered.
2  time ← time +1
3  d[u] ← time                  // record the discovery time
4  for each v ∈ Adj[u]          ▹Explore edge(u, v).
5       do if color[v] = WHITE
6             then π[v] ← u      // set the parent value
7                  DFS-VISIT(v)          // recursive call
8  color[u] BLACK       ▹ Blacken u; it is finished.
9  f [u] ▹ time ← time +1
```

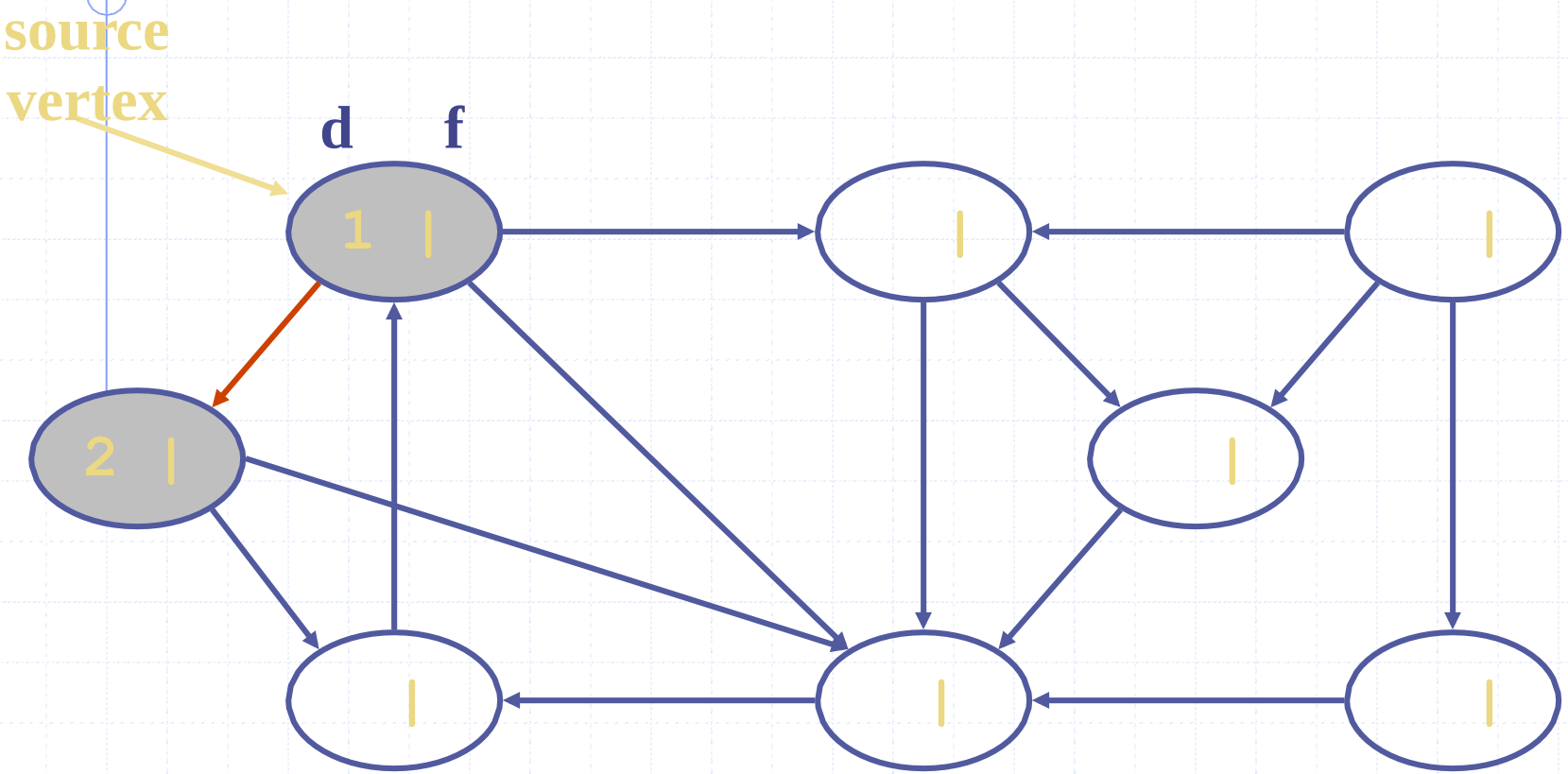
DFS Example



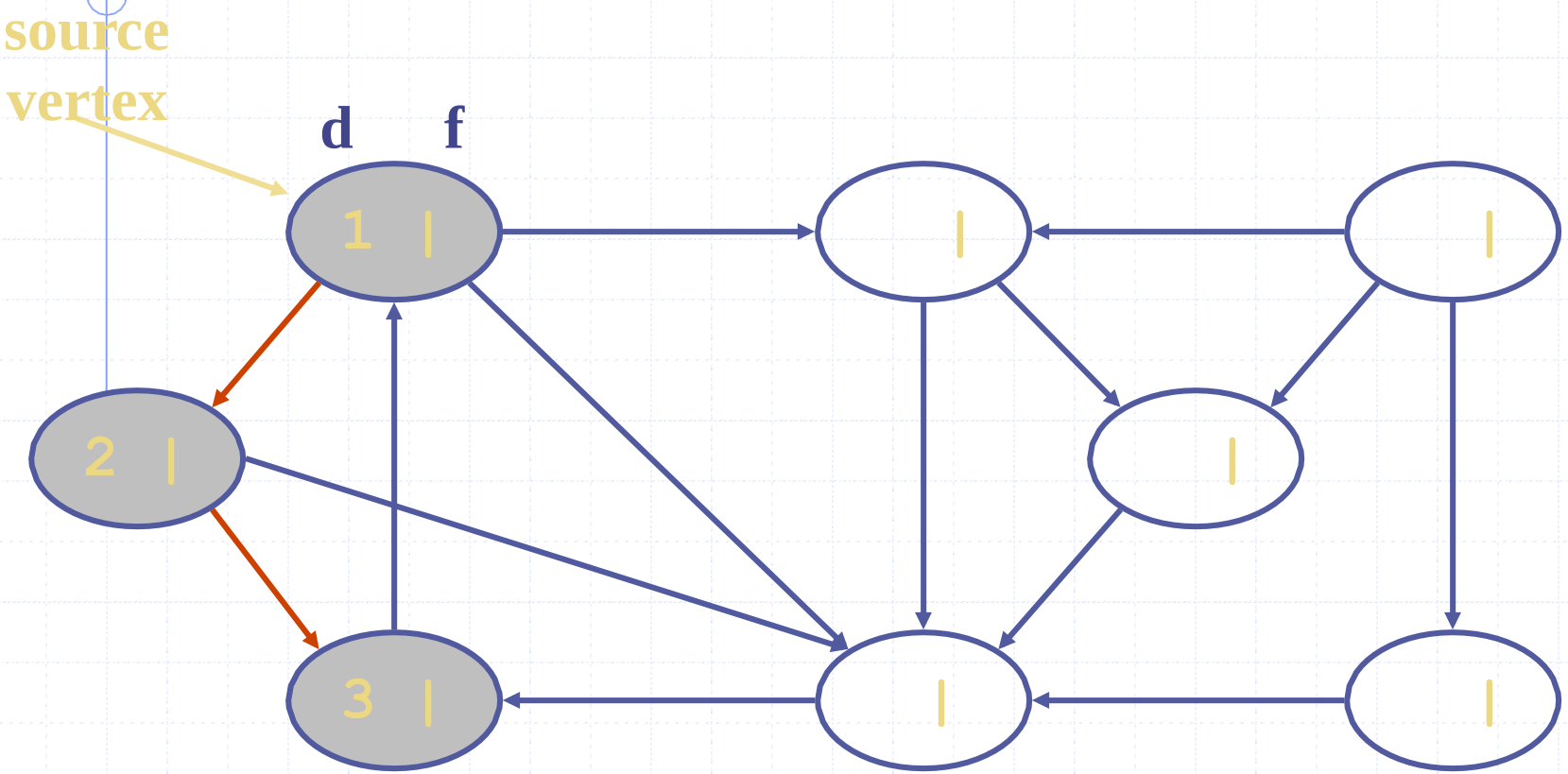
DFS Example



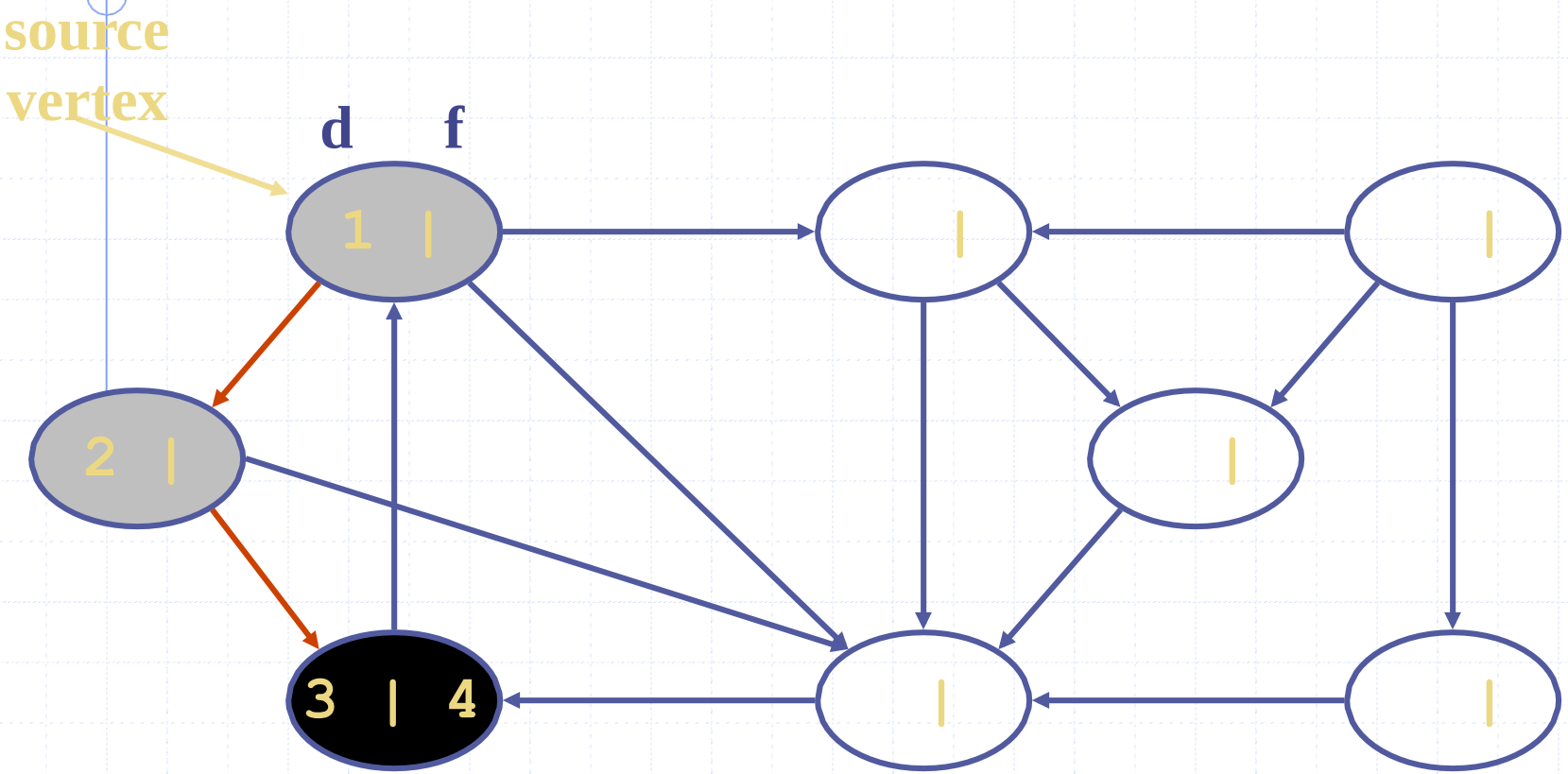
DFS Example



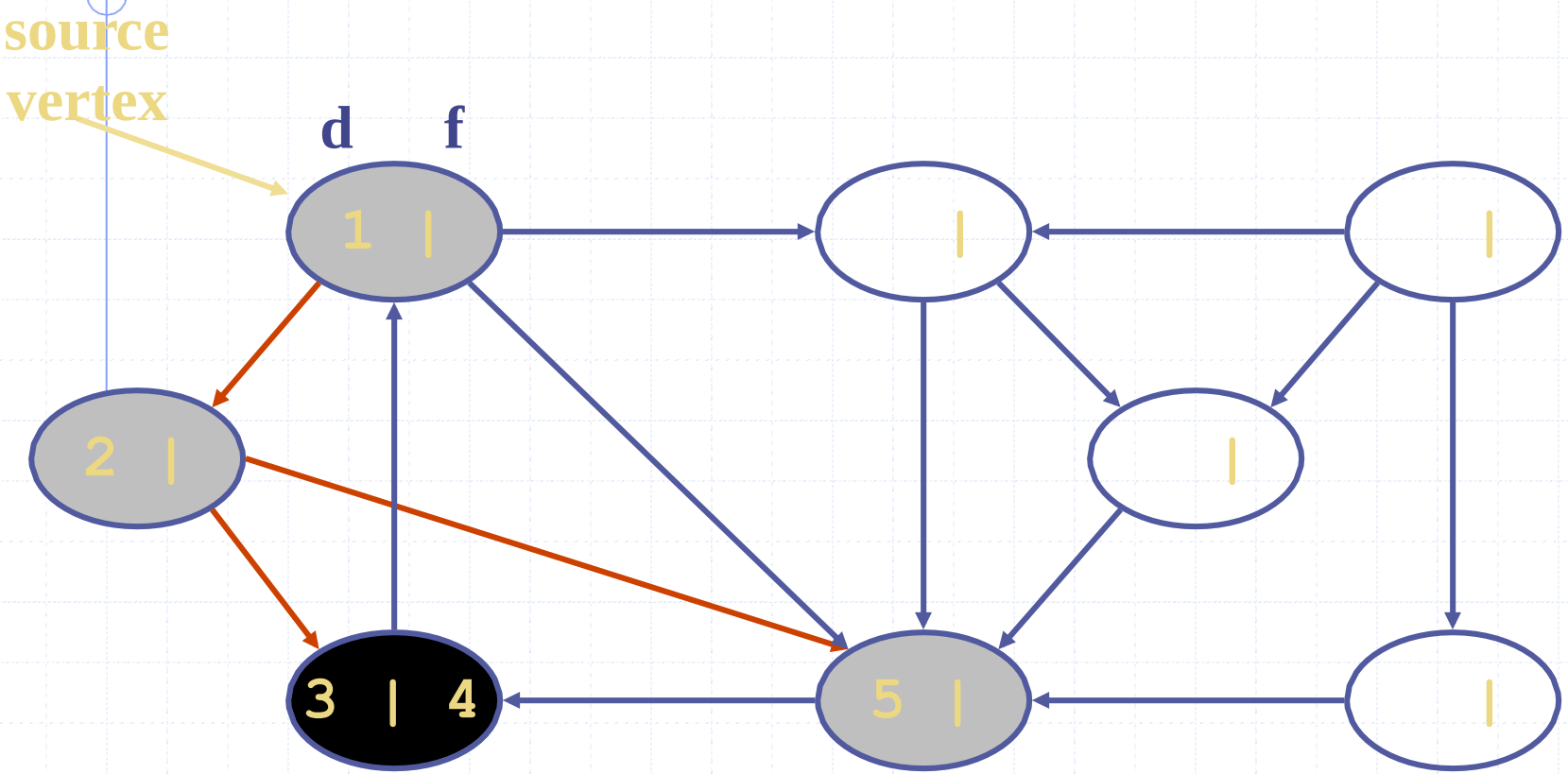
DFS Example



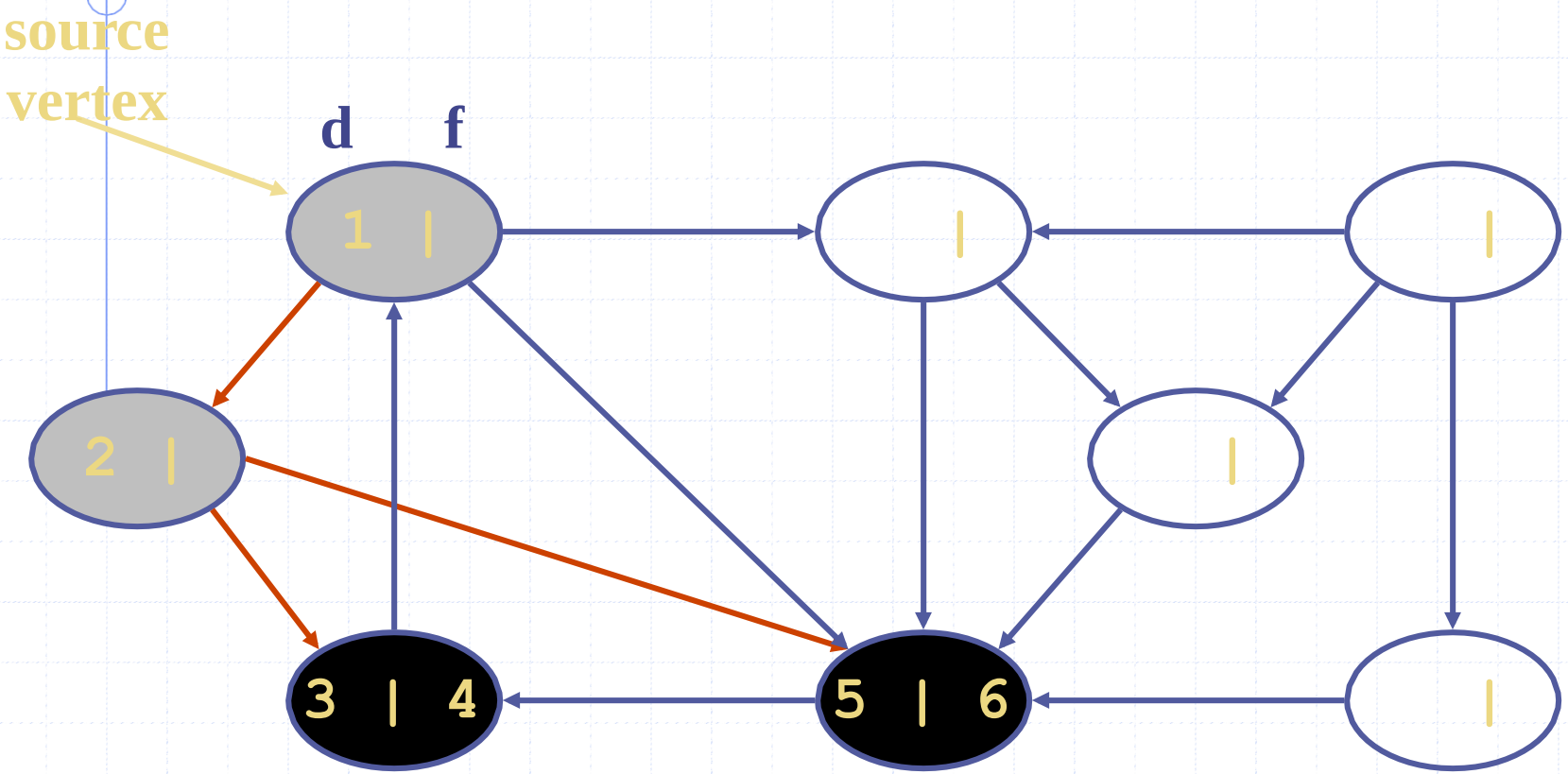
DFS Example



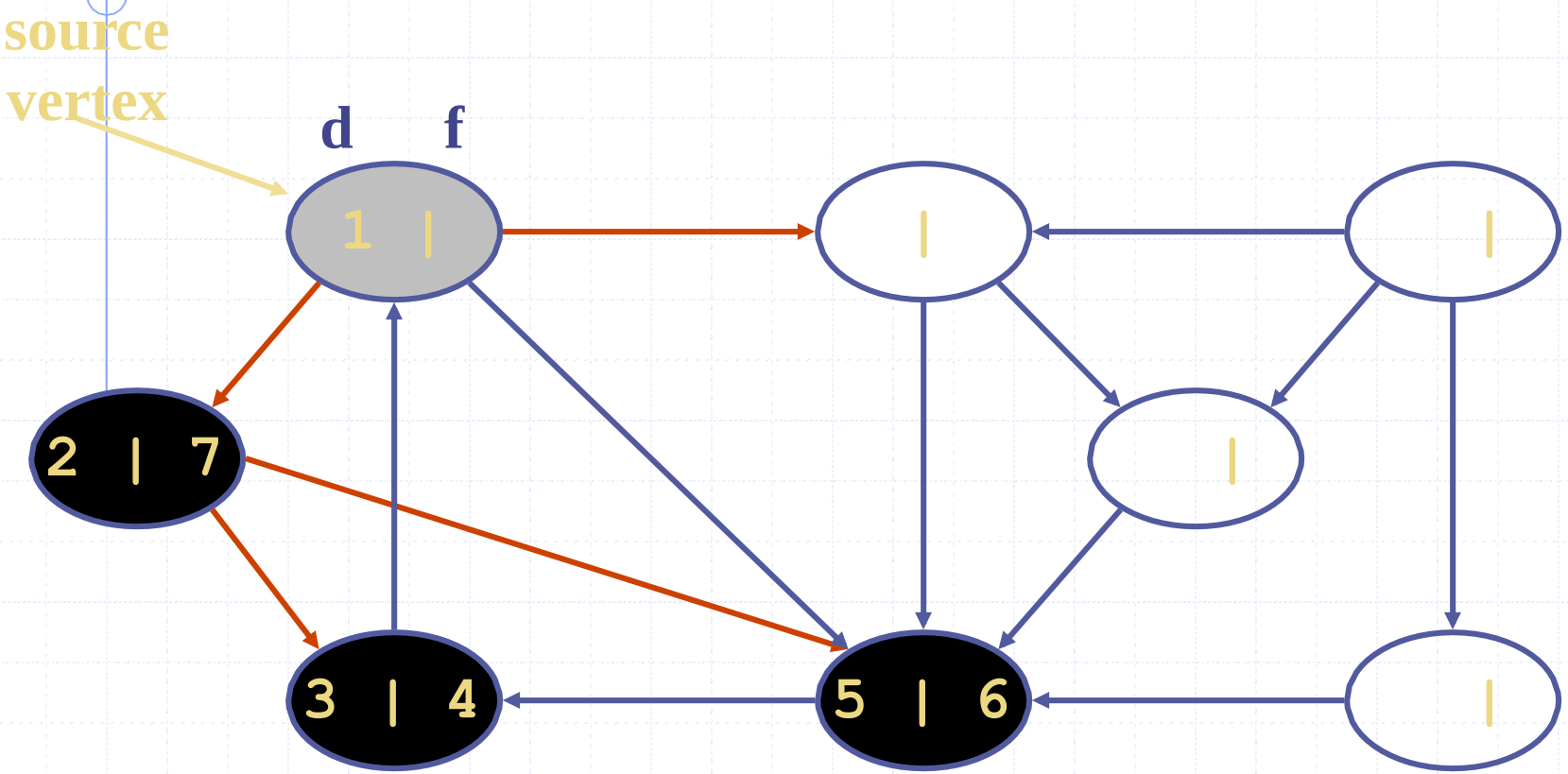
DFS Example



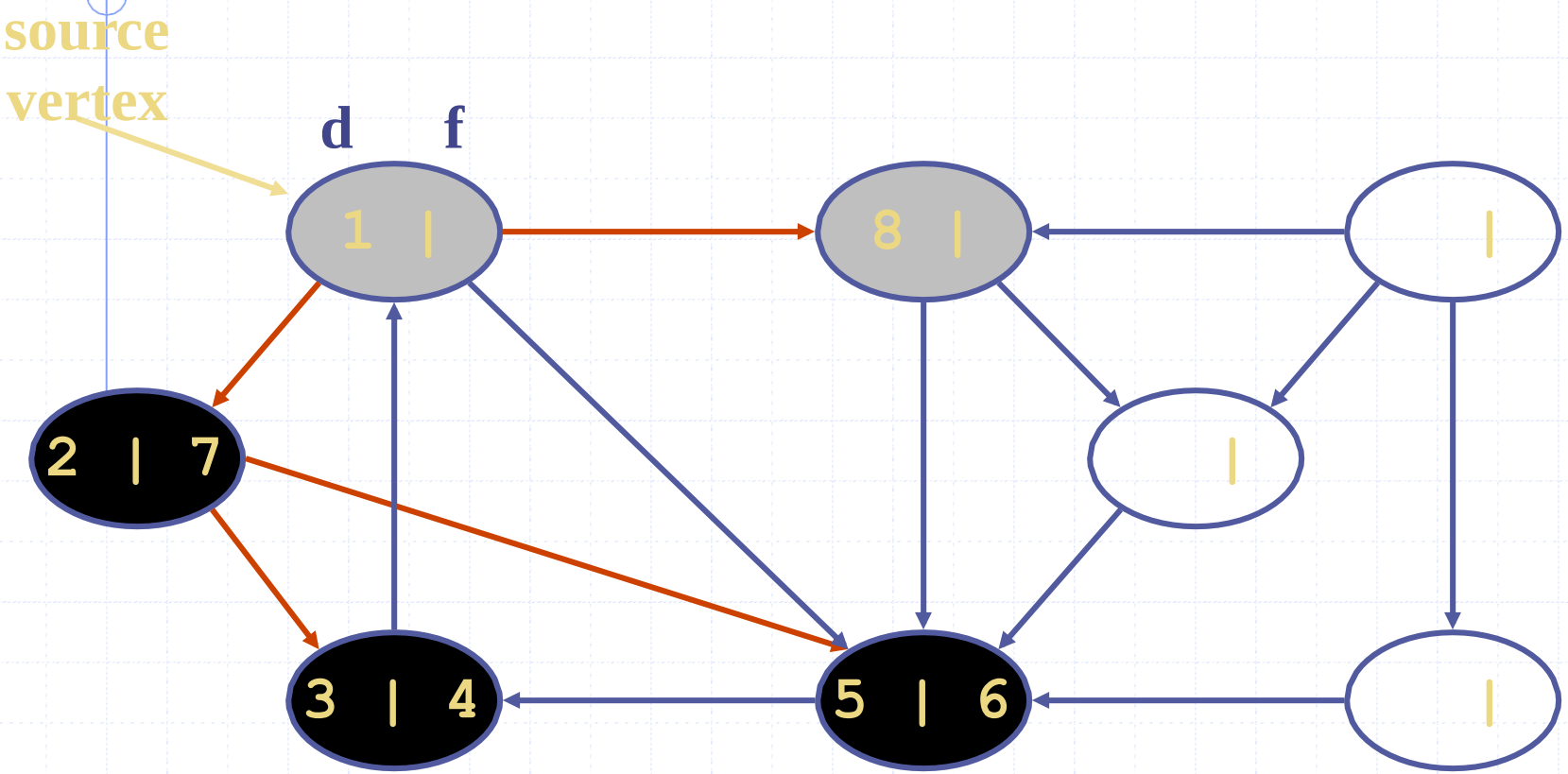
DFS Example



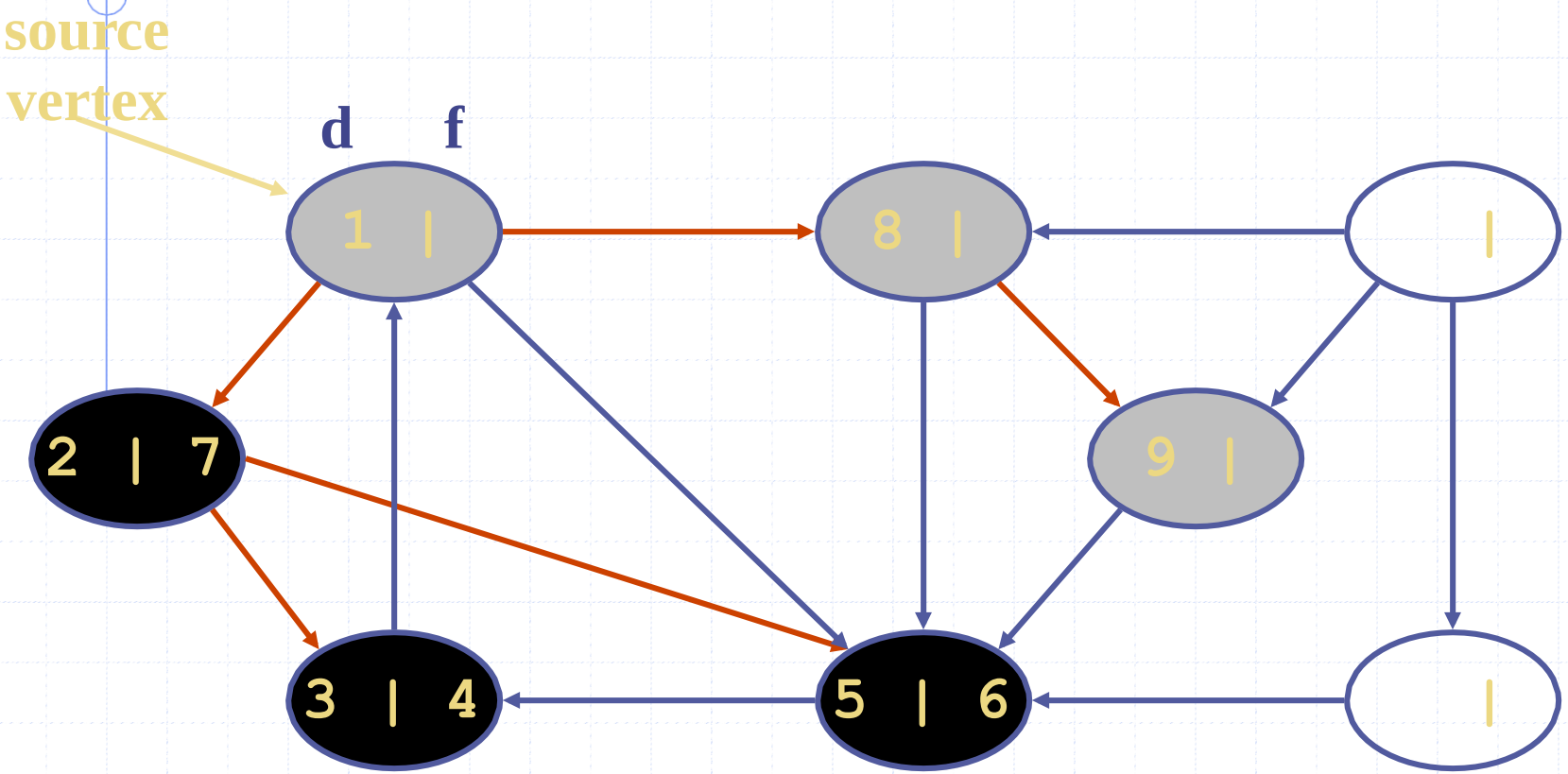
DFS Example



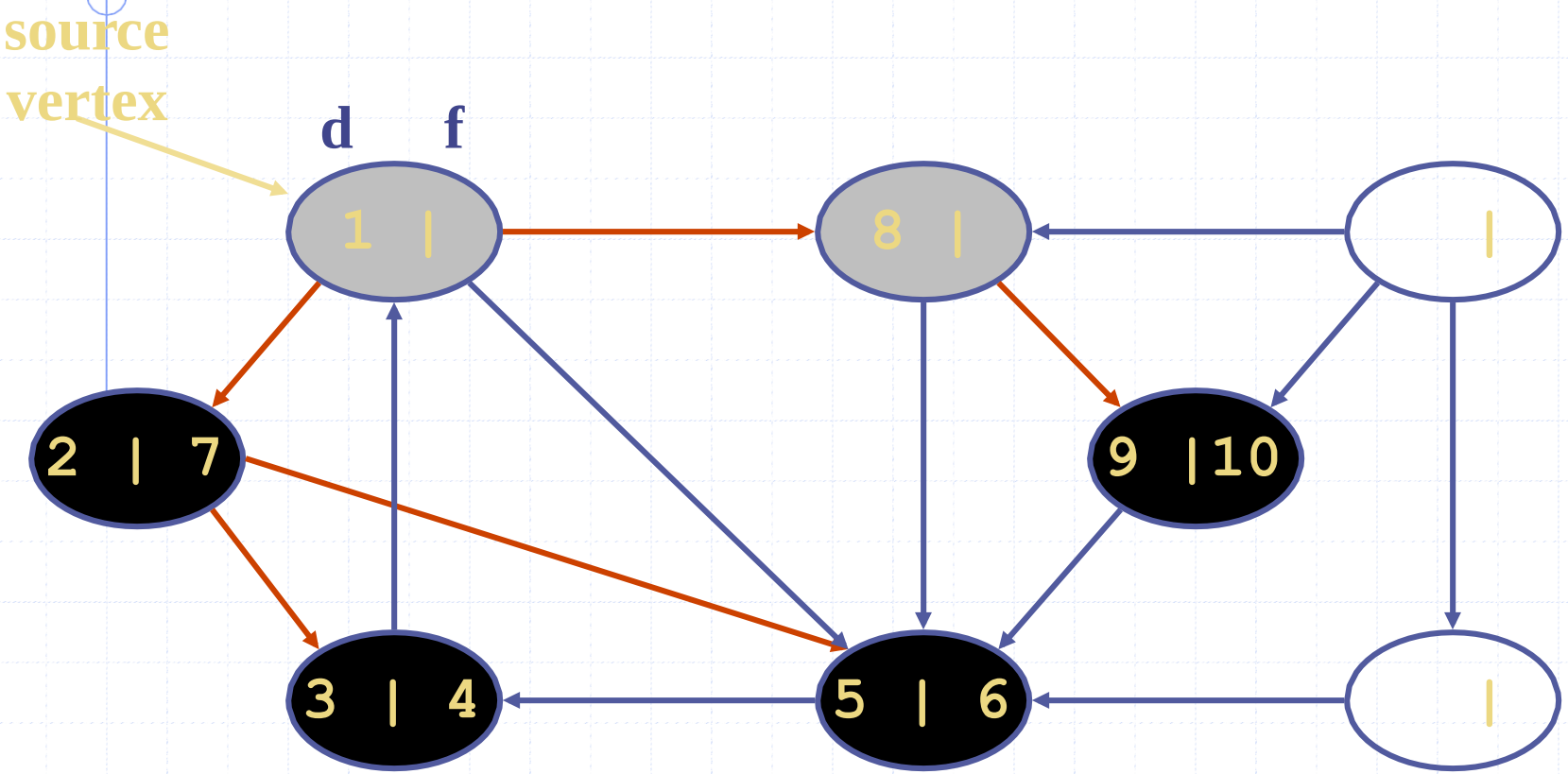
DFS Example



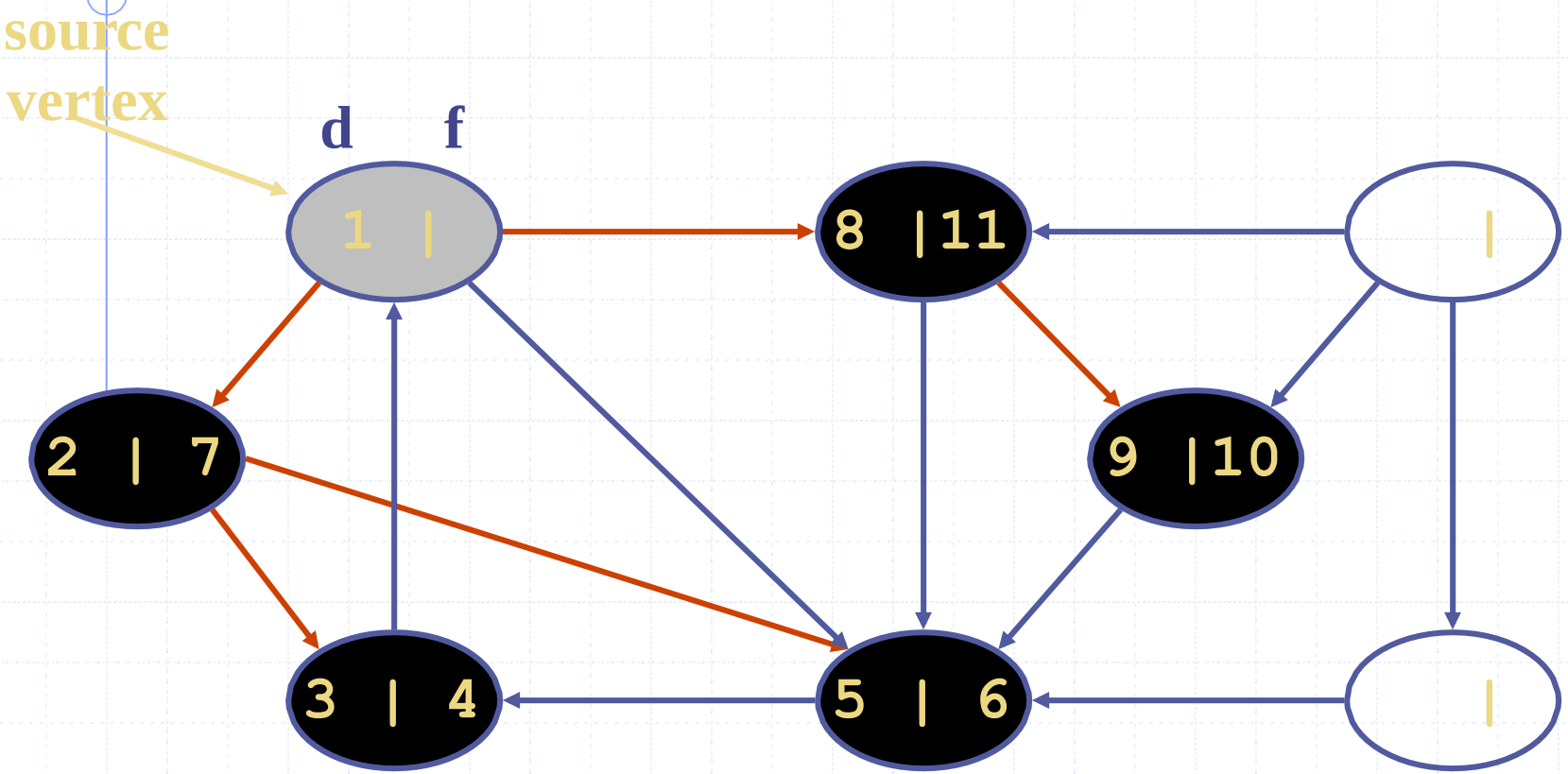
DFS Example



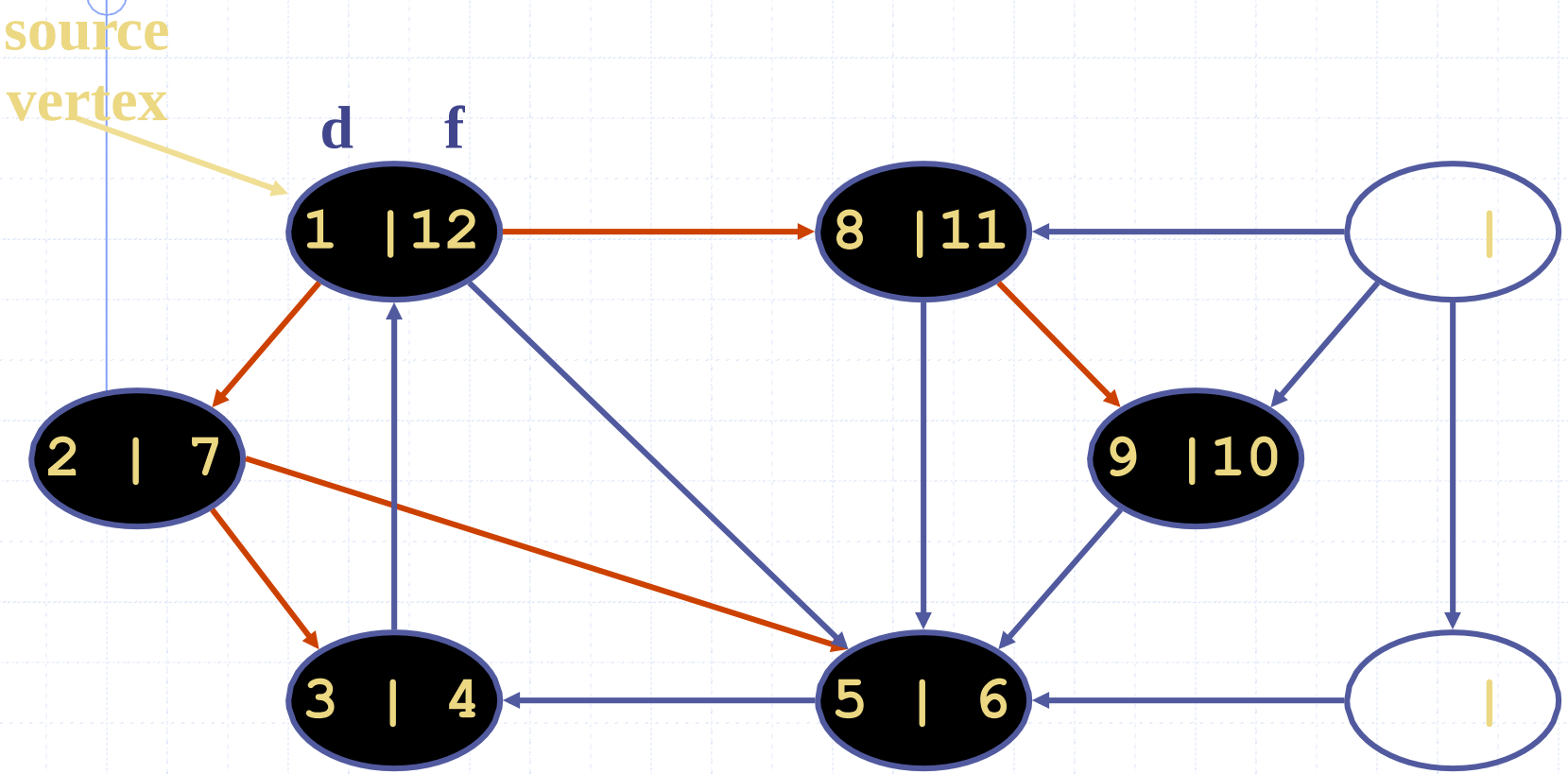
DFS Example



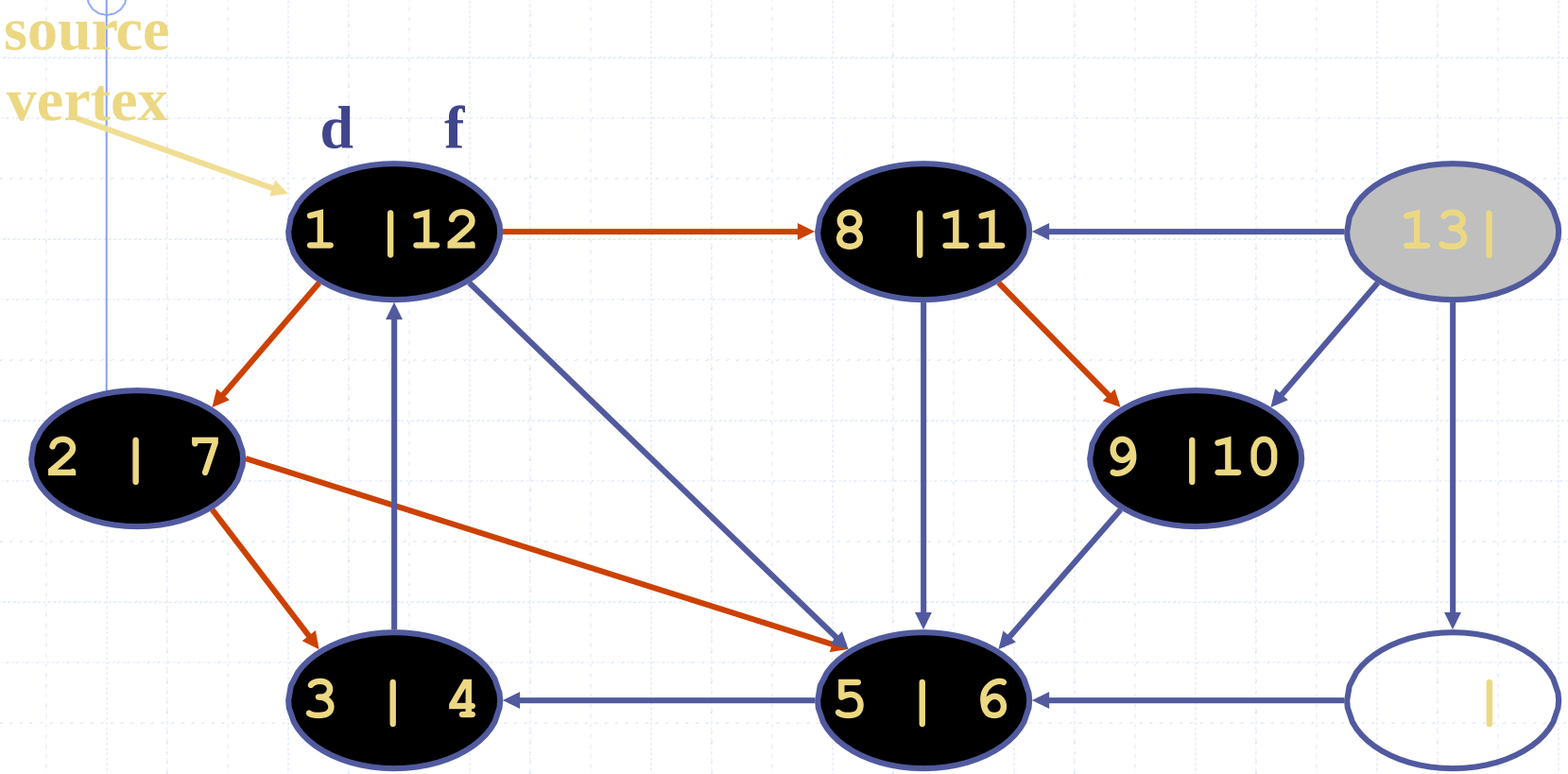
DFS Example



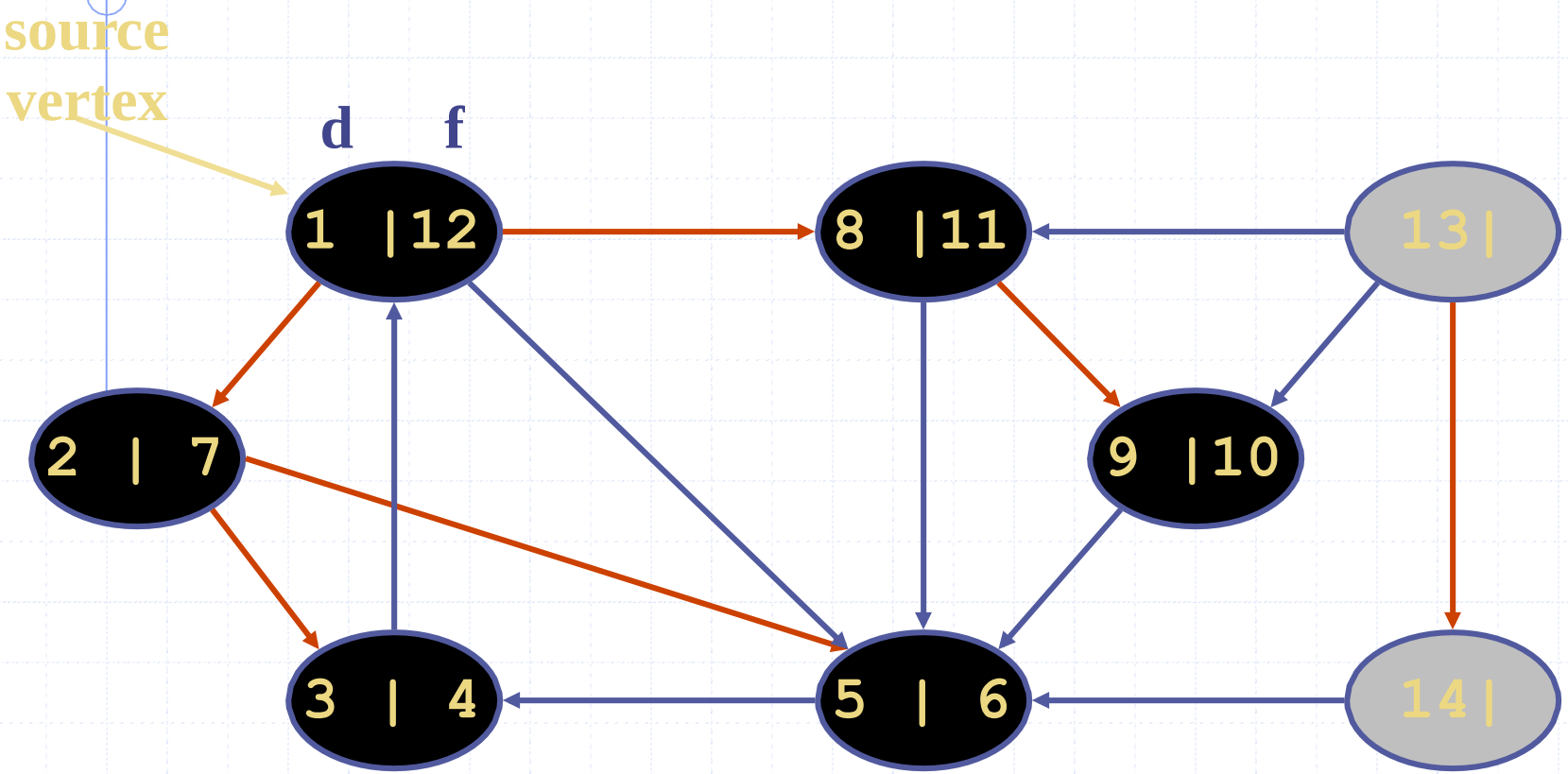
DFS Example



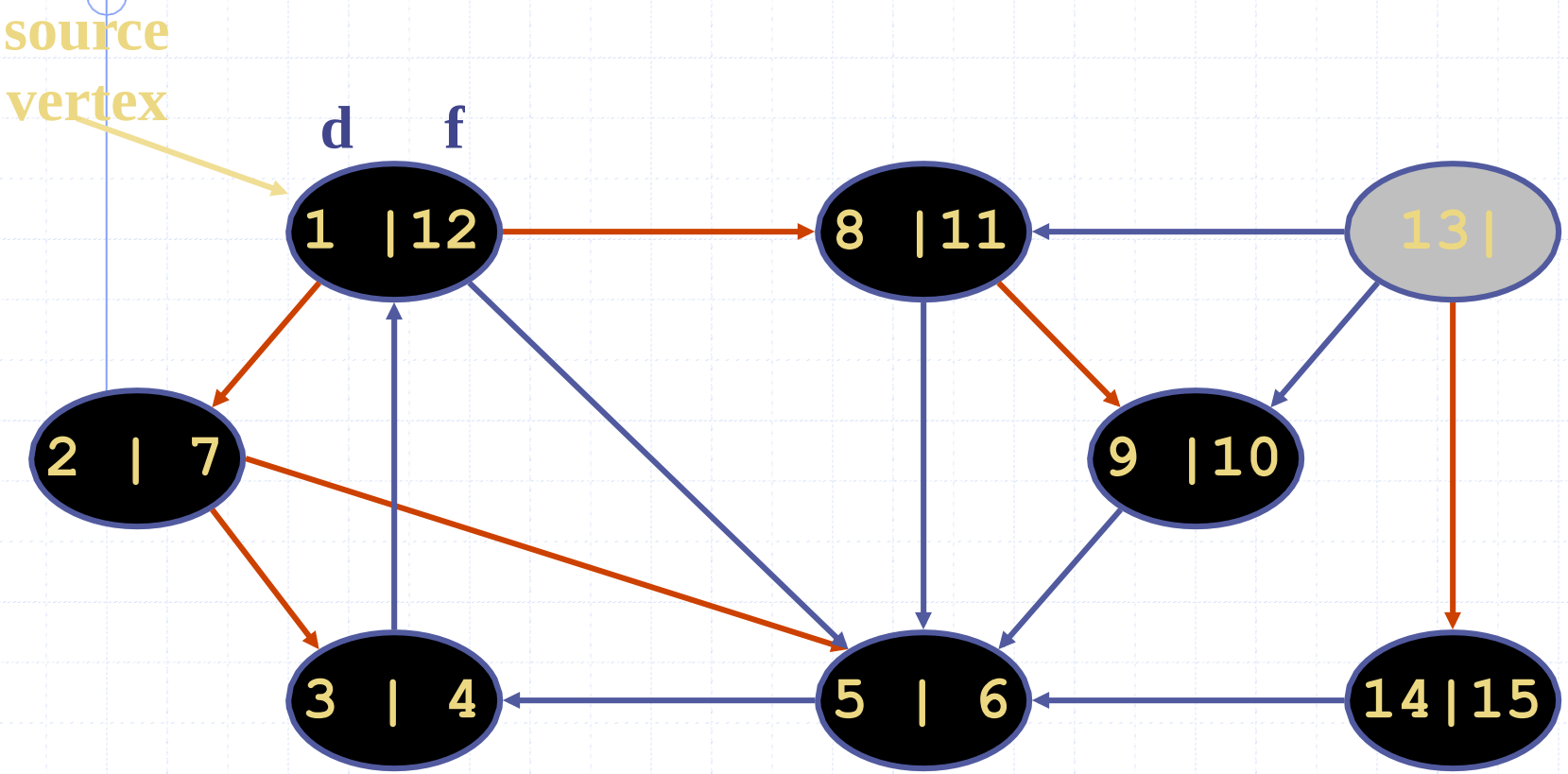
DFS Example



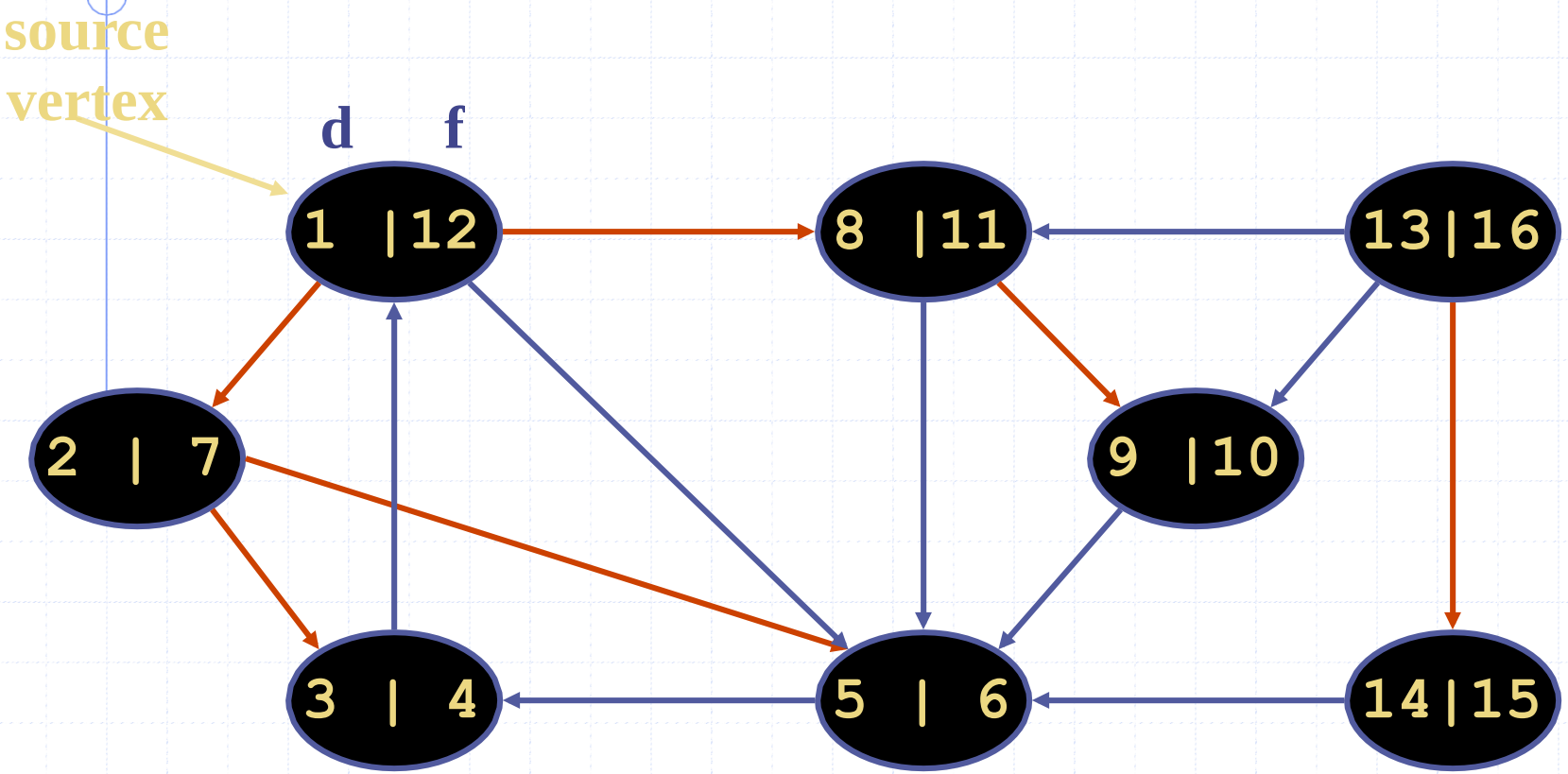
DFS Example



DFS Example



DFS Example



Depth first search - analysis

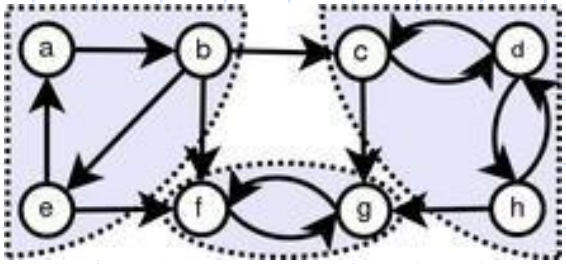
- Lines 1-3, initialization take time $\Theta(V)$.
- Lines 5-7 take time $\Theta(V)$, excluding the time to call the DFS-VISIT.
- DFS-VISIT is called only once for each node (since it's called only for white nodes and the first step in it is to paint the node gray).
- Loop on line 4-7 is executed $|Adj(v)|$ times. Since, $\Sigma_{v \in V} |Adj(v)| = \Theta(E)$, the total cost of DFS-VISIT it $\Theta(E)$.
- The total cost of DFS is $\Theta(V+E)$.

BFS and DFS - comparison

- Space complexity of DFS is lower than that of BFS.
- Time complexity of both is same – O(|V|+|E|).
- The behavior differs for graphs where not all the vertices can be reached from the given vertex s.
- Predecessor sub-graphs produced by DFS may be different than those produced by BFS. The BFS product is just one tree whereas the DFS product may be multiple trees.
- BFS can generate minimum spanning tree.

Strongly Connected Components

- One of the applications of DFS.
- Used to partition a graph into its strongly connected components.



- A strongly connected component of a directed graph G (V,E) is a maximal set of vertices $C \subseteq V$ such that for every pair of vertices u and v in C, we have both $u \rightsquigarrow v$ and $v \rightsquigarrow u$; that is, vertices u and v are reachable from each other.